EECS 498/CSE 598: Zero-Knowledge Proofs
Winter 2026
Lecture 20

Paul Grubbs

paulgrub@umich.edu

Beyster 4709

Agenda for this lecture

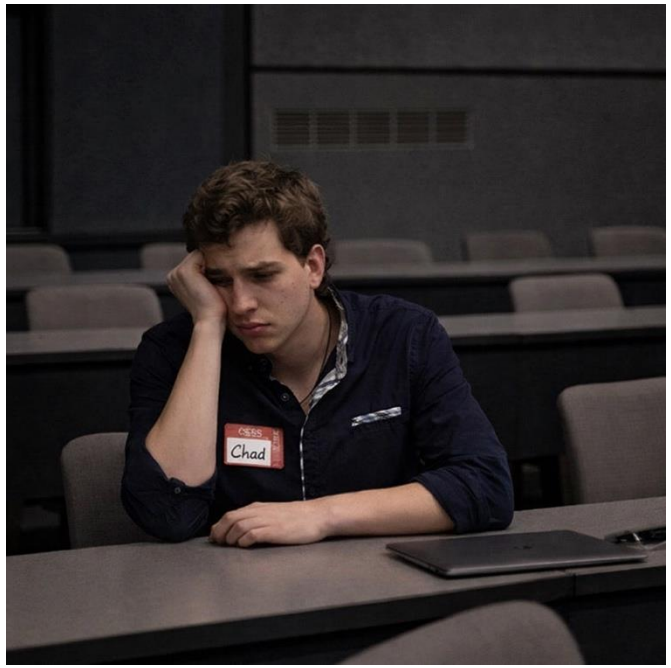
- Announcements
- Motivating incremental proofs
- Incrementally verifiable computation (IVC) via recursion
- IVC via folding, the Nova construction
- Proof-carrying data

Agenda for this lecture

- Announcements
- Motivating incremental proofs
- Incrementally verifiable computation (IVC) via recursion
- IVC via folding, the Nova construction
- Proof-carrying data

Announcements

- The plan is to release P4 and theory pset 2 ASAP.
  - Note there will be no P5 nor a final exam. These will be the last assignments of the semester.
- 598 research track: **project proposal due Friday 3/27**
- All work for this course must be completed by our final exam date, April 27th
- Reminder: you can get participation credit for going to discussion.
  - Chad is getting lonely on Fridays!



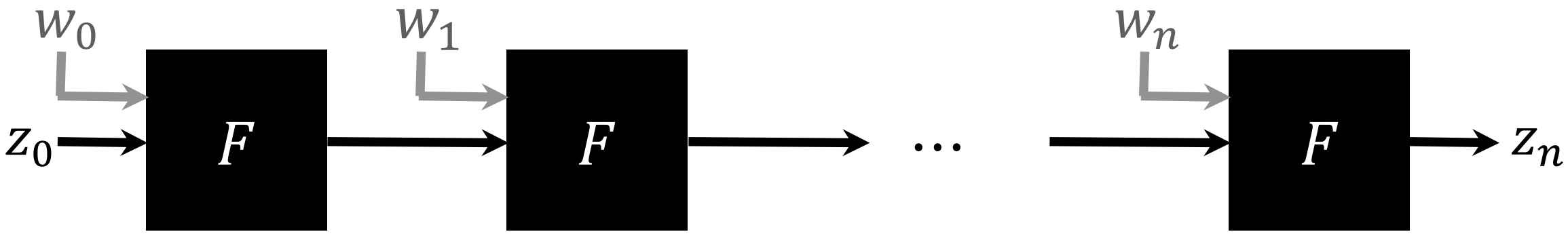
Agenda for this lecture

- Announcements
- Motivating incremental proofs
- Incrementally verifiable computation (IVC) via recursion
- IVC via folding, the Nova construction
- Proof-carrying data

Most of the slides in this lecture are due to Abhiram Kothapalli

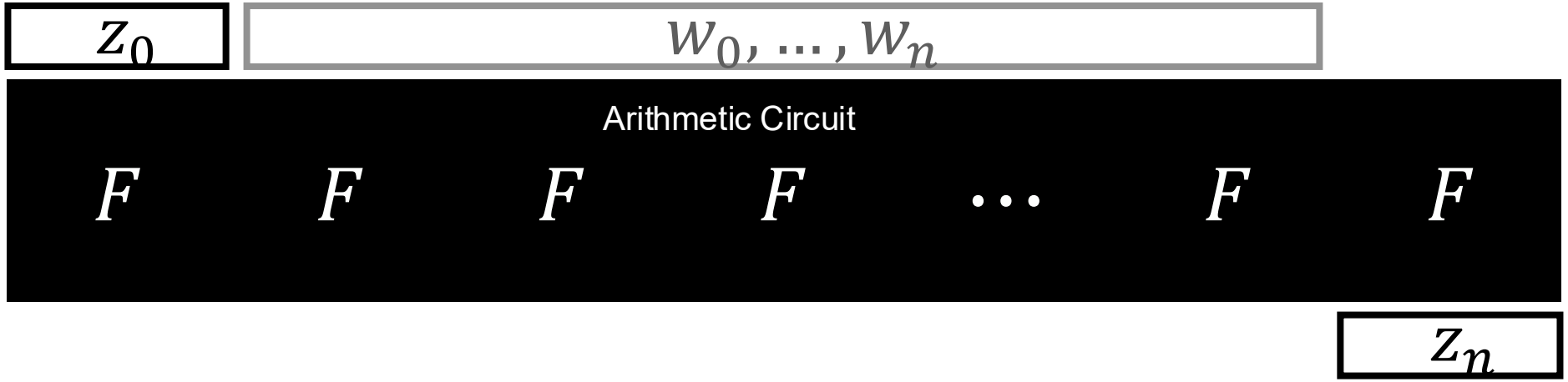
Goal: Practical SNARKs for Recursive Computation

Proving VM execution reduces to proving that function F applied n times to initial input z_0 results in z_n



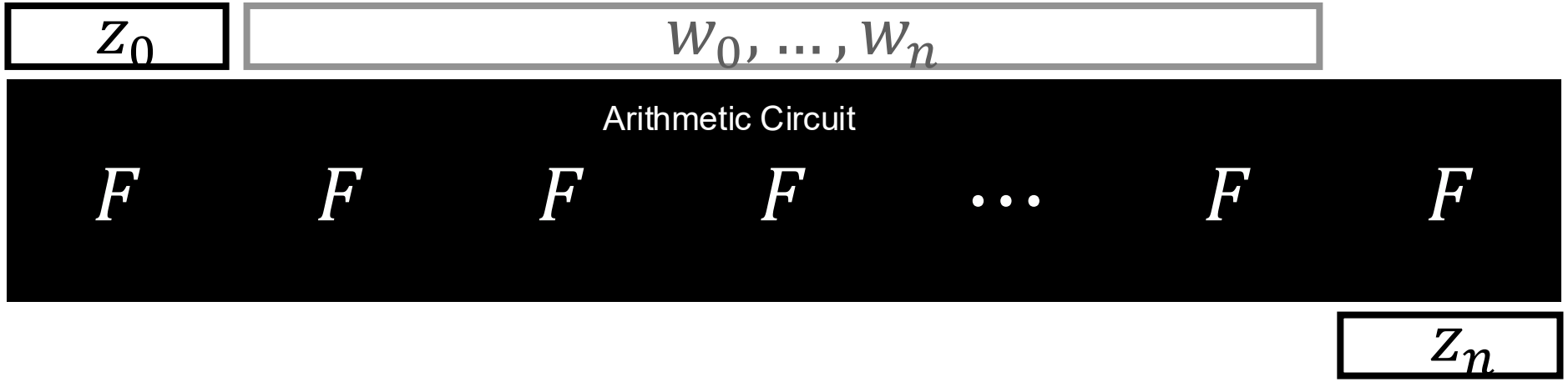
Naive Approach

Use a SNARK for Circuit-SAT to monolithically prove the unrolled statement



Naive Approach

Use a SNARK for Circuit-SAT to monolithically prove the unrolled statement



Drawbacks

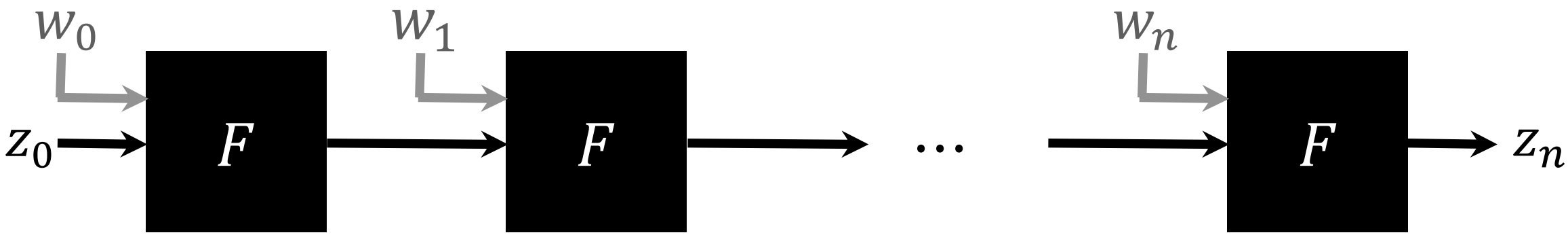
- Fixes n ahead of time
- $O(n \cdot |F|)$ prover memory and verifier preprocessing time
- Verifier time may depend on n

Agenda for this lecture

- Announcements
- Motivating incremental proofs
- Incrementally verifiable computation (IVC) via recursion
- IVC via folding, the Nova construction
- Proof-carrying data

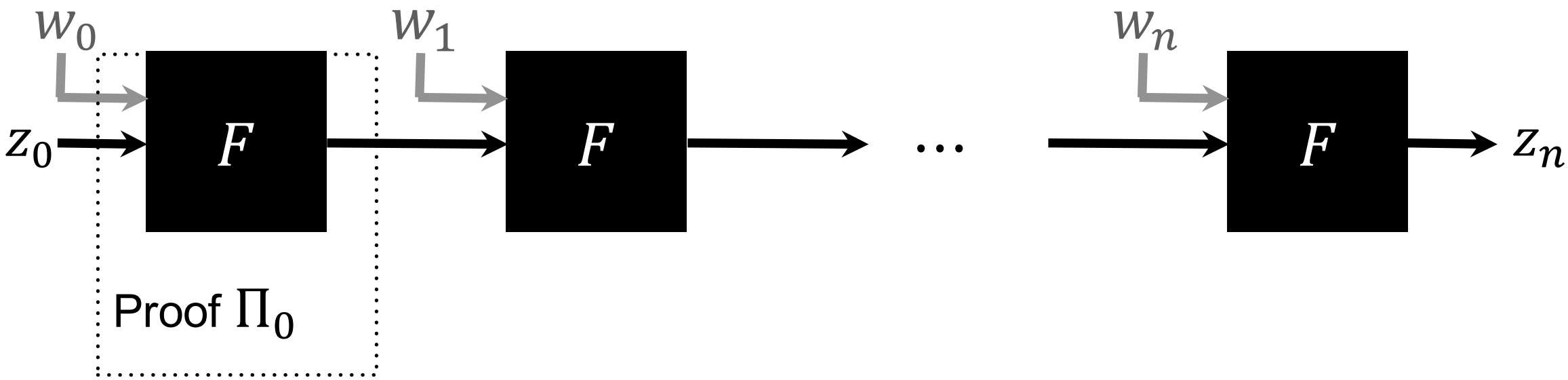
Approach: Incrementally Verifiable Computation (IVC) [Val08]

Incrementally update a proof of i applications to a proof of $i + 1$ applications with the same size



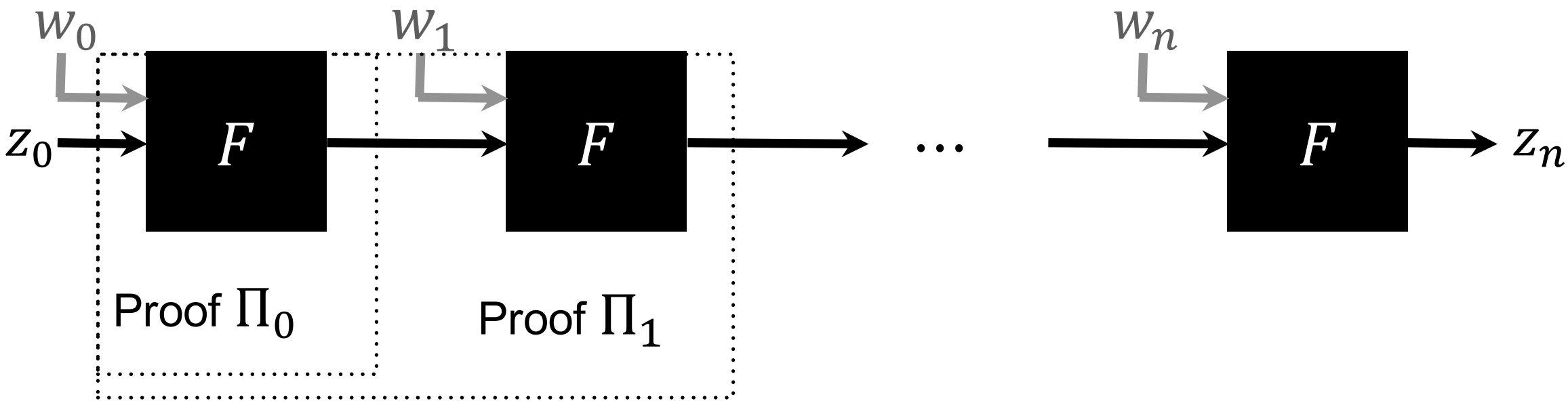
Approach: Incrementally Verifiable Computation (IVC) [Val08]

Incrementally update a proof of i applications to a proof of $i + 1$ applications with the same size



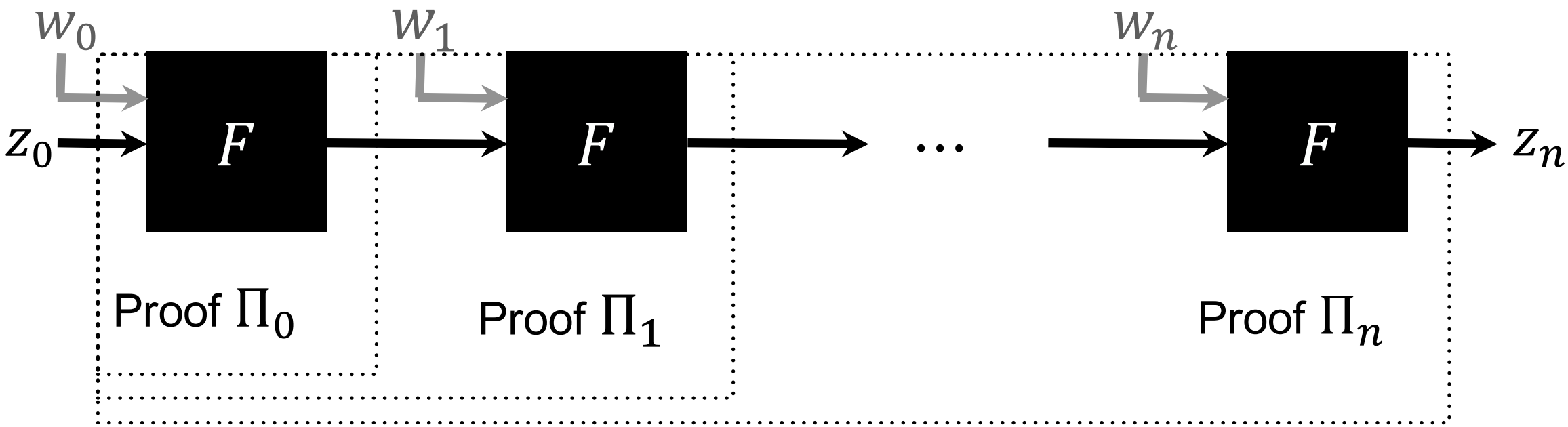
Approach: Incrementally Verifiable Computation (IVC) [Val08]

Incrementally update a proof of i applications to a proof of $i + 1$ applications with the same size

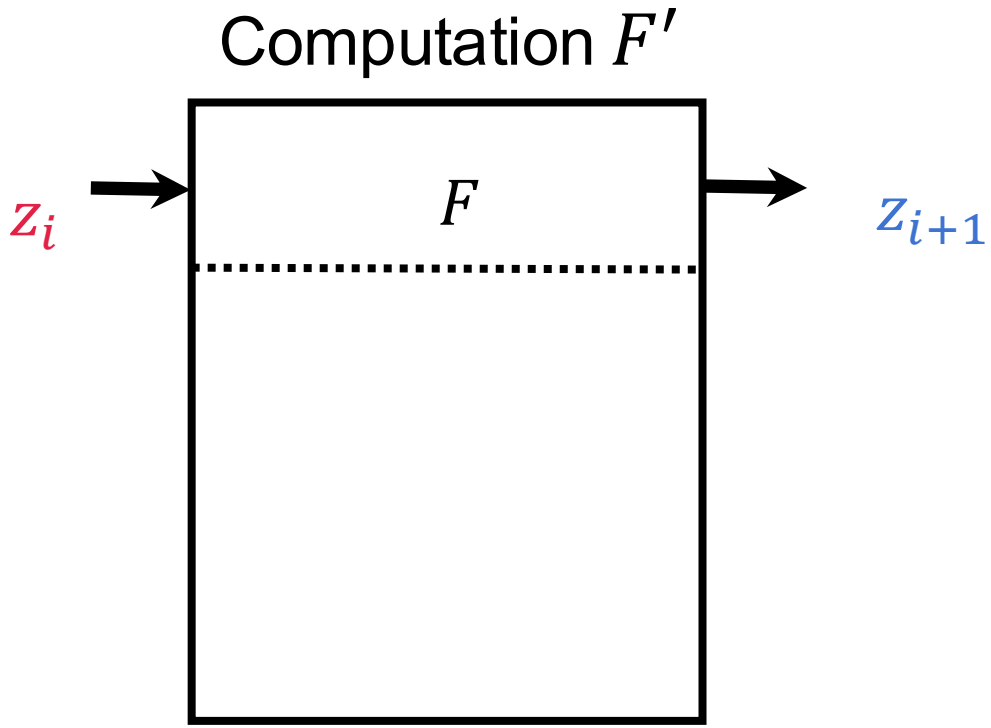


Approach: Incrementally Verifiable Computation (IVC) [Val08]

Incrementally update a proof of i applications to a proof of $i + 1$ applications with the same size

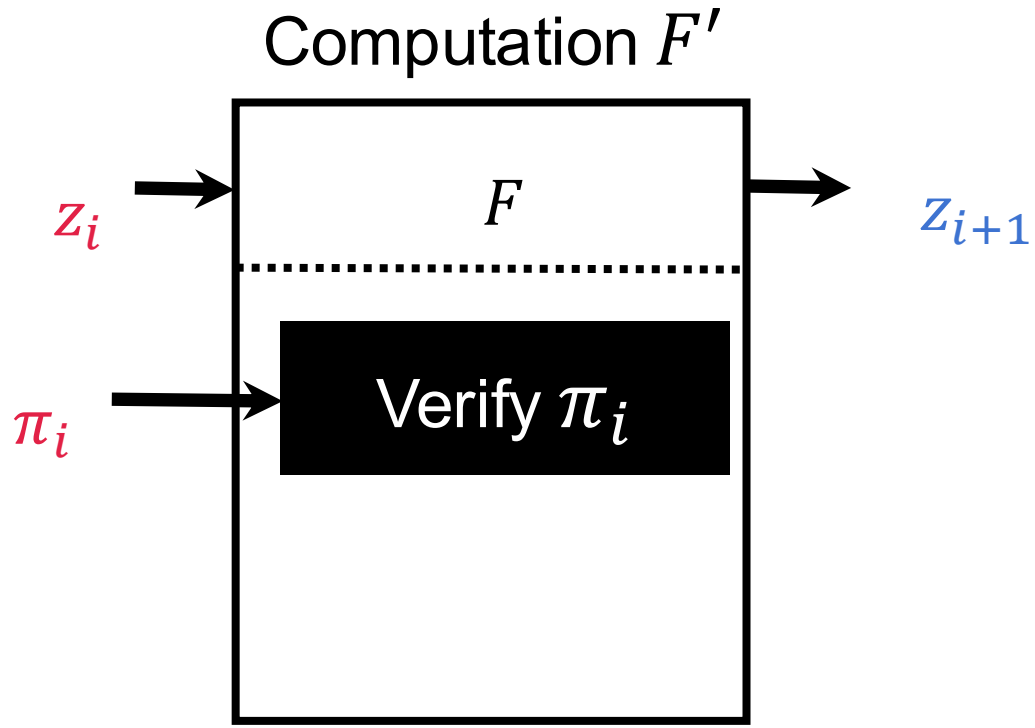


Valiant's Original IVC Approach [Val08,BCTV14]

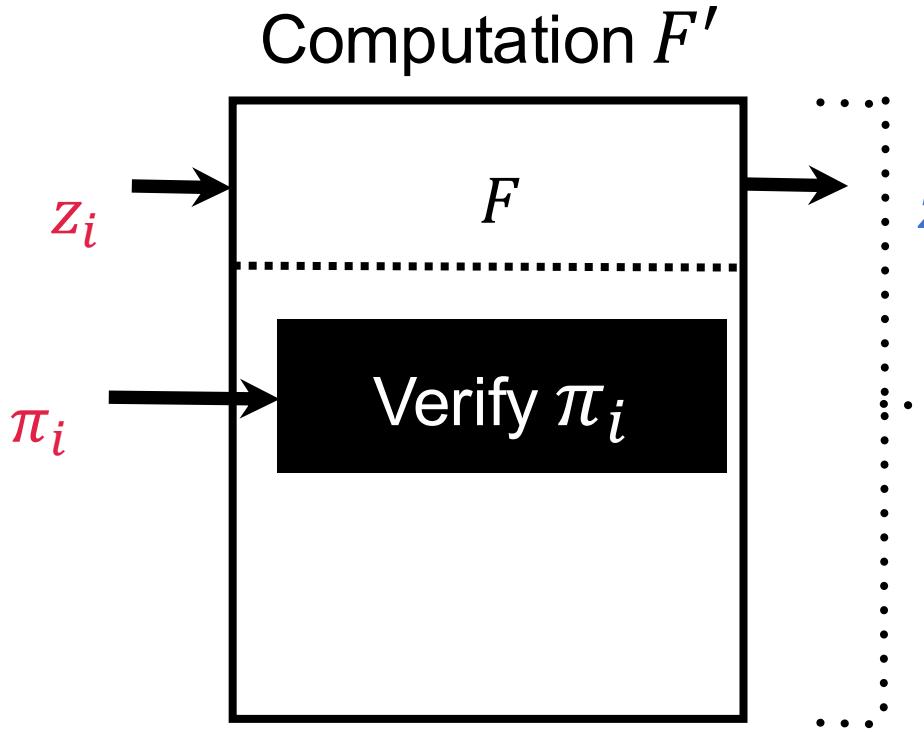


Valiant's Original IVC Approach [Val08,BCTV14]

SNARK Proof of
$(F')^i(z_0) = z_i$



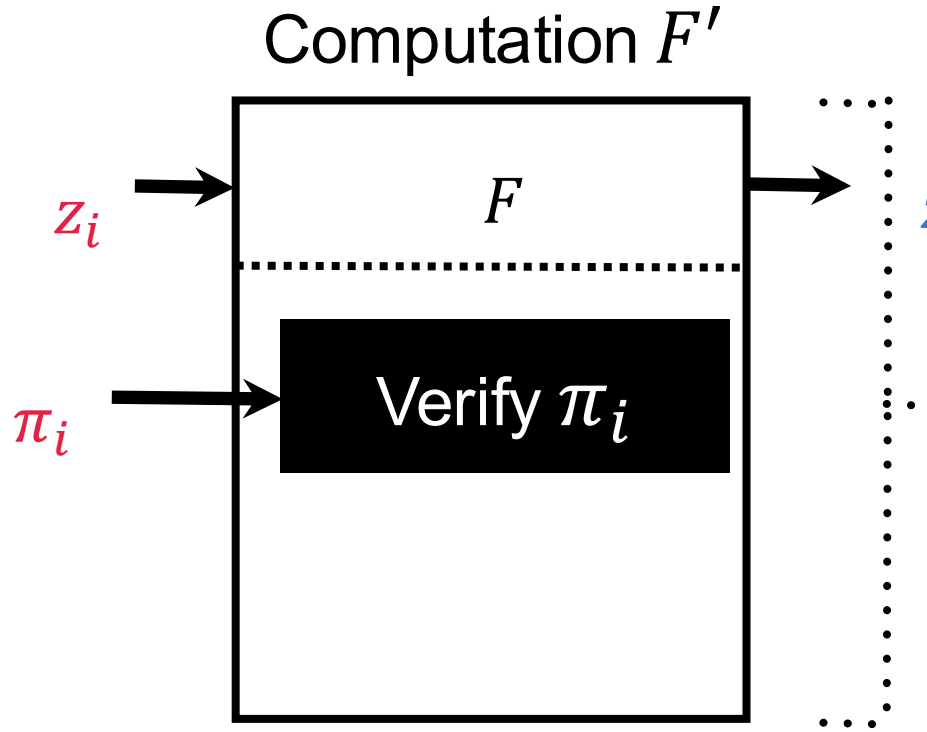
Valiant's Original IVC Approach [Val08,BCTV14]



SNARK Proof of
$(F')^i(z_0) = z_i$

SNARK Proof of
$(F')^{i+1}(z_0) = z_{i+1}$

Valiant's Original IVC Approach [Val08,BCTV14]



SNARK Proof of $(F')^i(z_0) = z_i$

SNARK Proof of $(F')^{i+1}(z_0) = z_{i+1}$

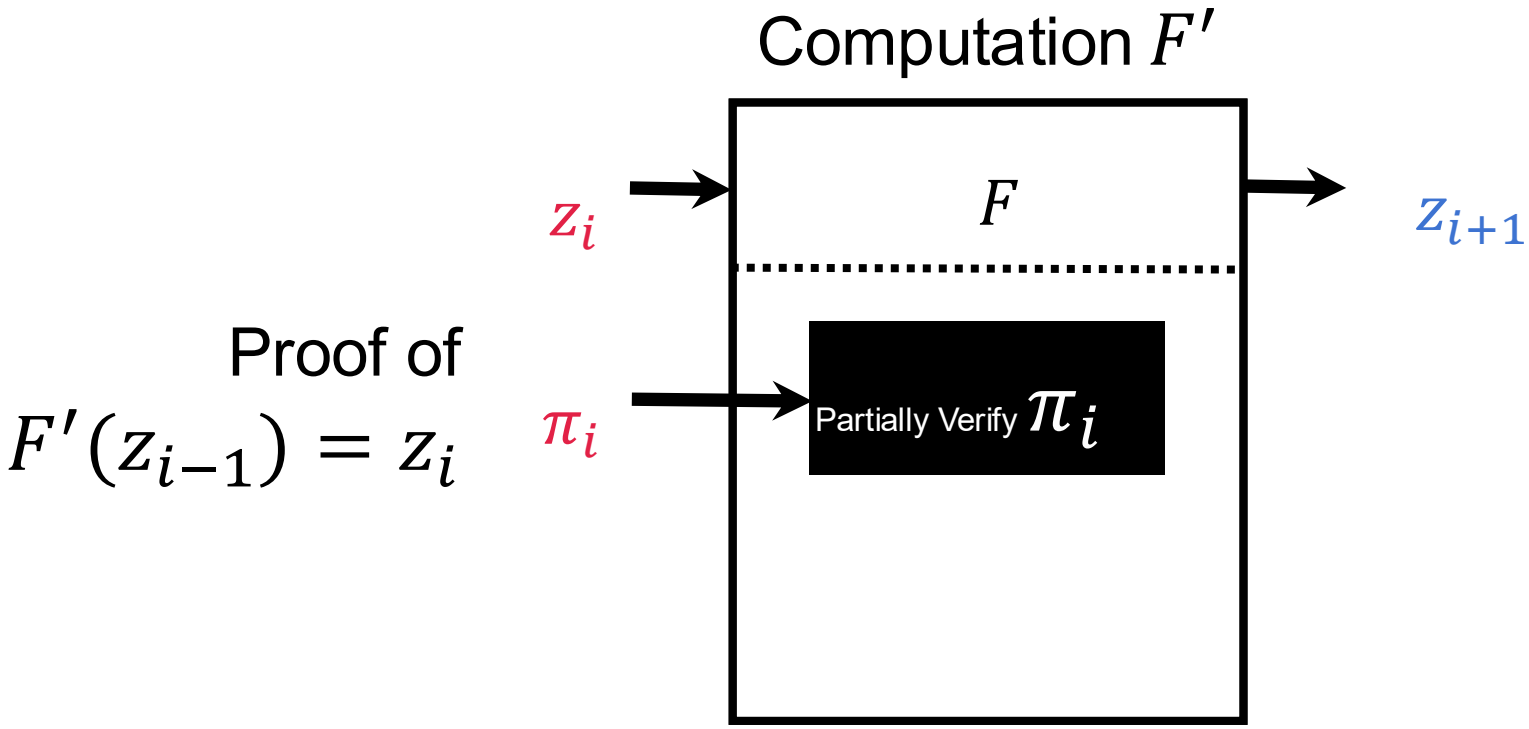
$|E/\mathbb{F}_p| = q$

$|E/\mathbb{F}_q| = p$

Drawbacks

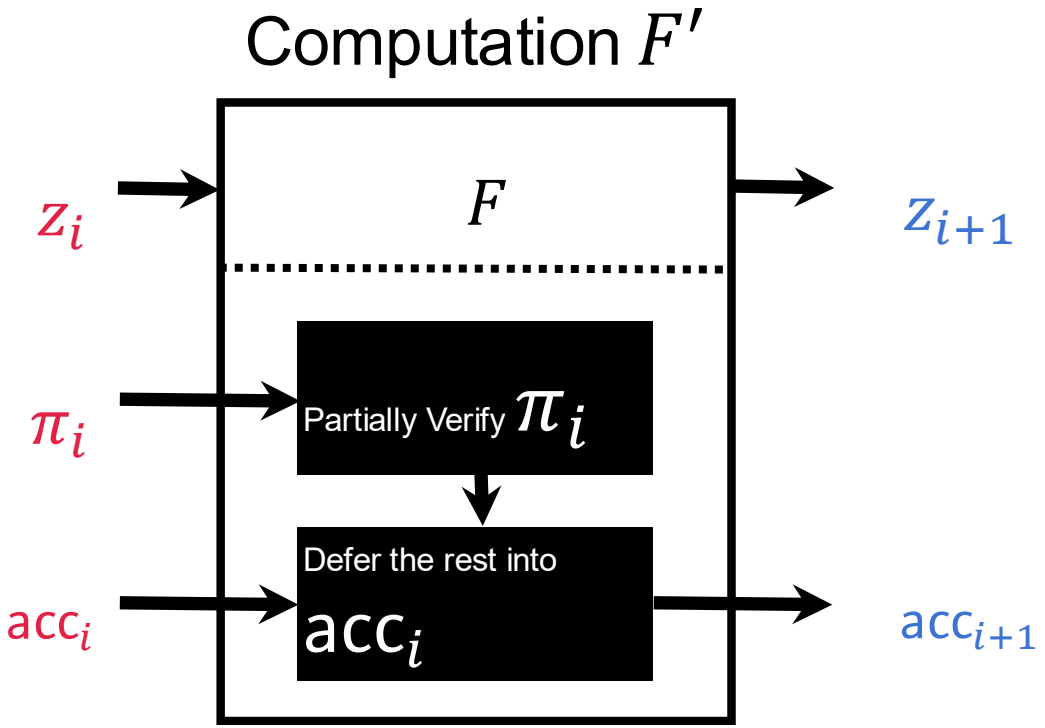
- Trusted-setup SNARKs require expensive cycles of pairing-friendly curves
- Utilizing SNARKs without trusted setup require much larger verifier circuits

Halo's Approach: Partially Verify Proofs [BGH19,BCLMS20,BDFG20]

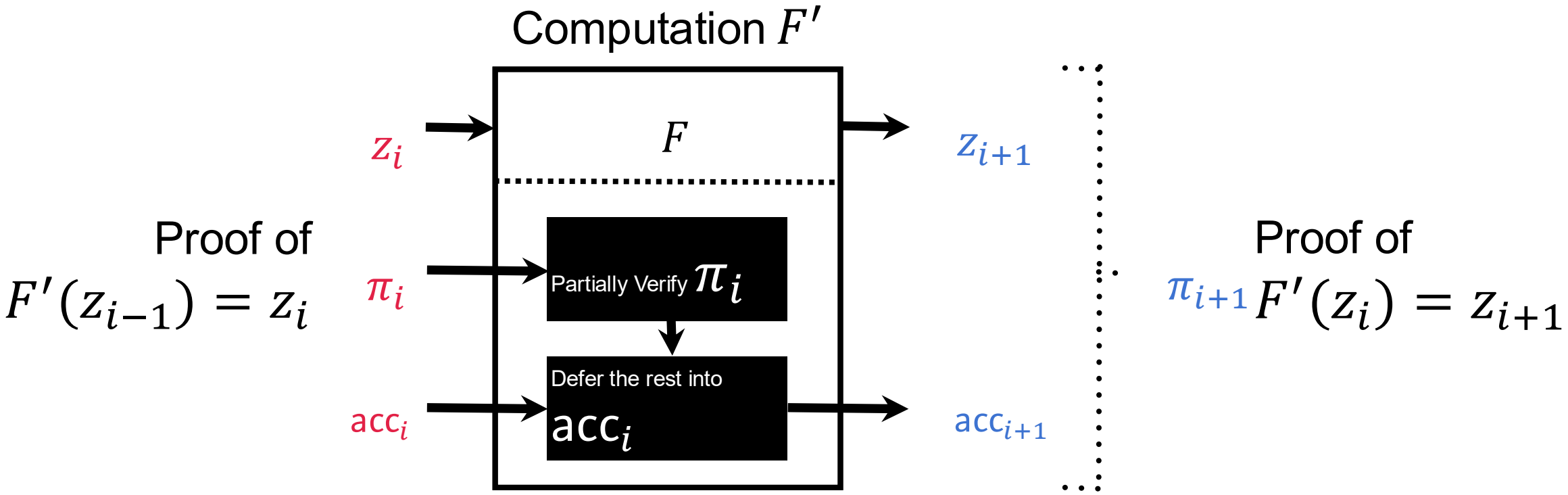


Halo's Approach: Partially Verify Proofs [BGH19,BCLMS20,BDFG20]

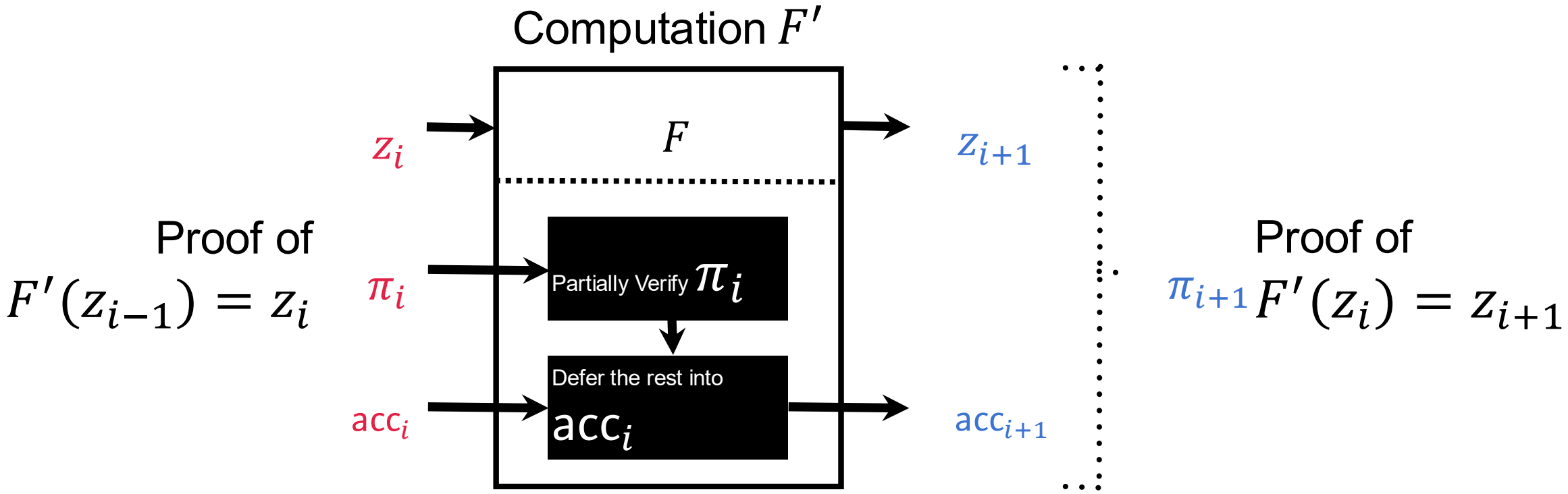
Proof of $F'(z_{i-1}) = z_i$



Halo's Approach: Partially Verify Proofs [BGH19,BCLMS20,BDFG20]



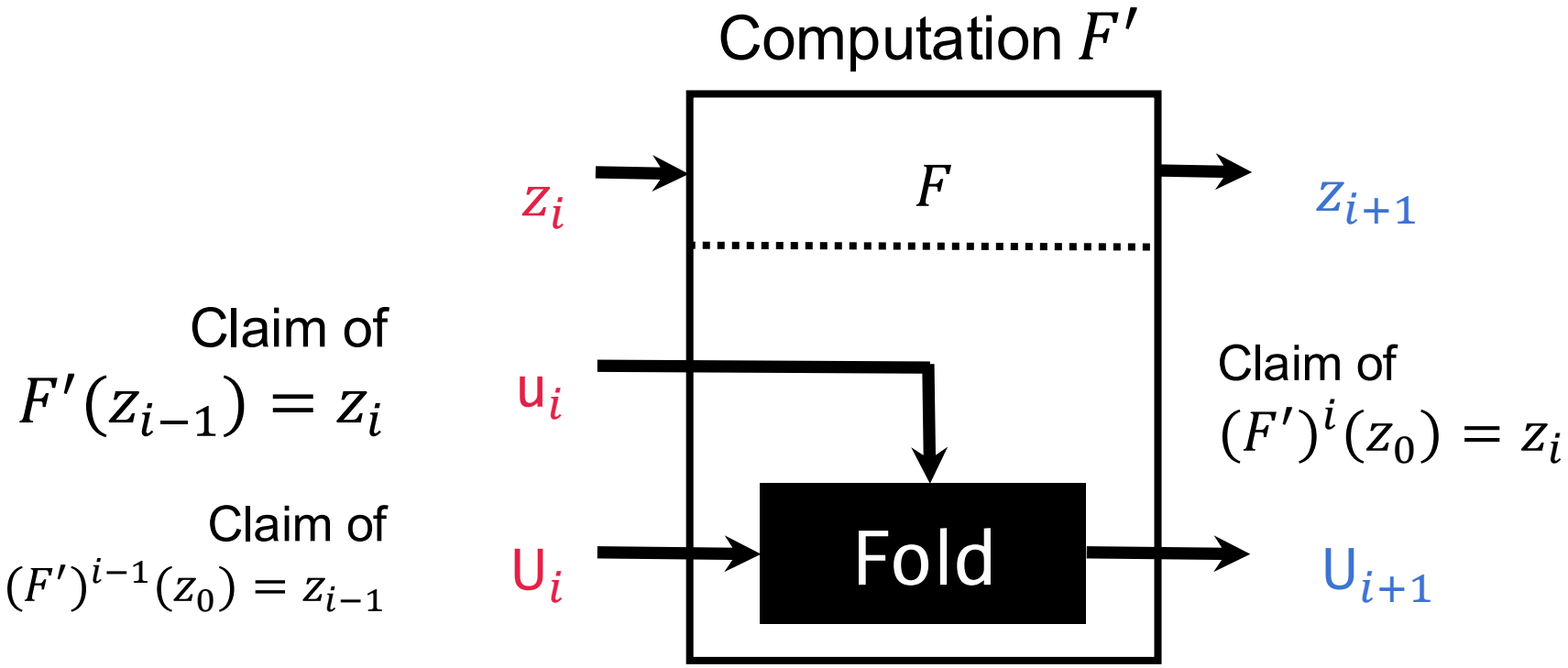
Halo's Approach: Partially Verify Proofs [BGH19,BCLMS20,BDFG20]



Drawbacks

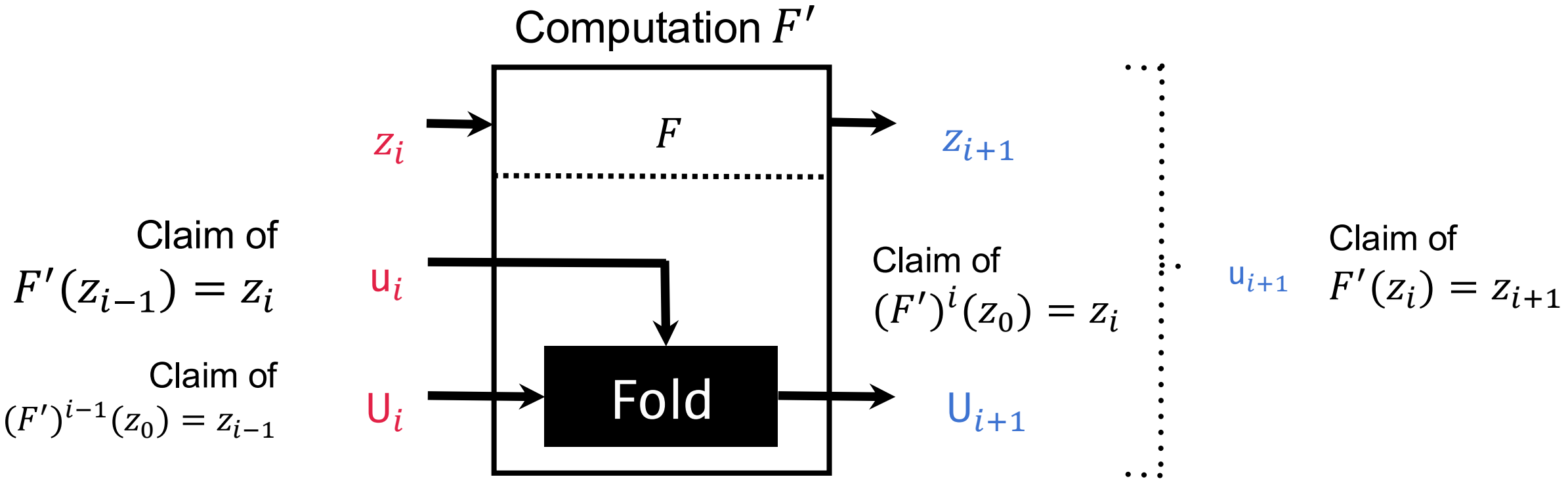
- Partially checking π_i in-circuit is still expensive
- Generating π_i is concretely and asymptotically expensive

Nova: Reduce Claims rather than Verify Proofs



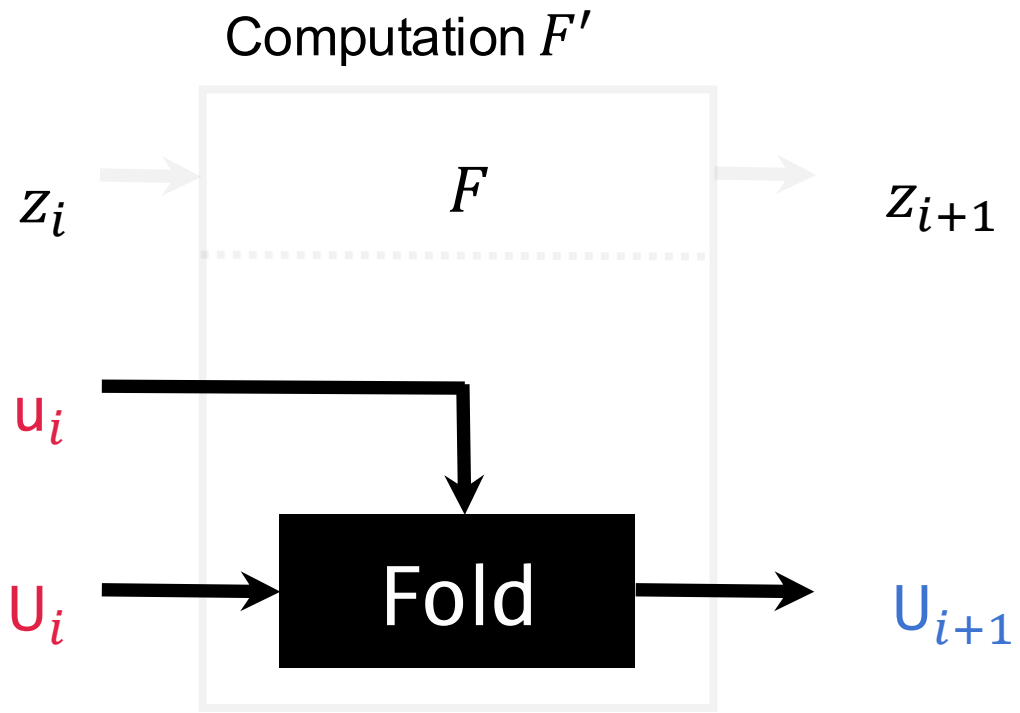
Fold reduces the task of checking two claims to the task of checking a single claim of the same size

Nova: Reduce Claims rather than Verify Proofs



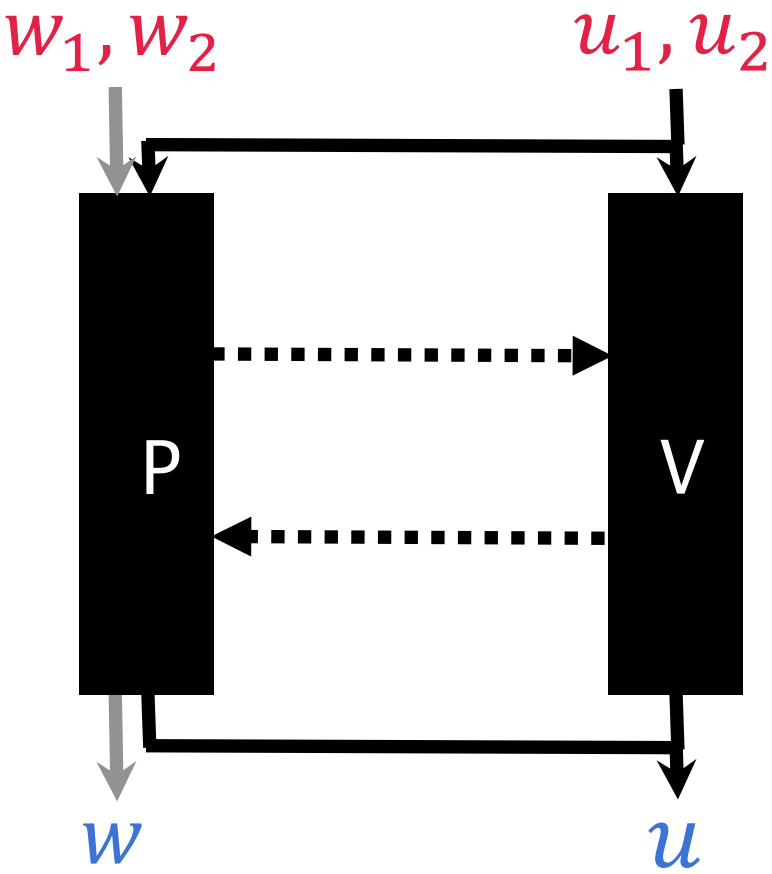
Fold reduces the task of checking two claims to the task of checking a single claim of the same size

How do we implement *Fold*?



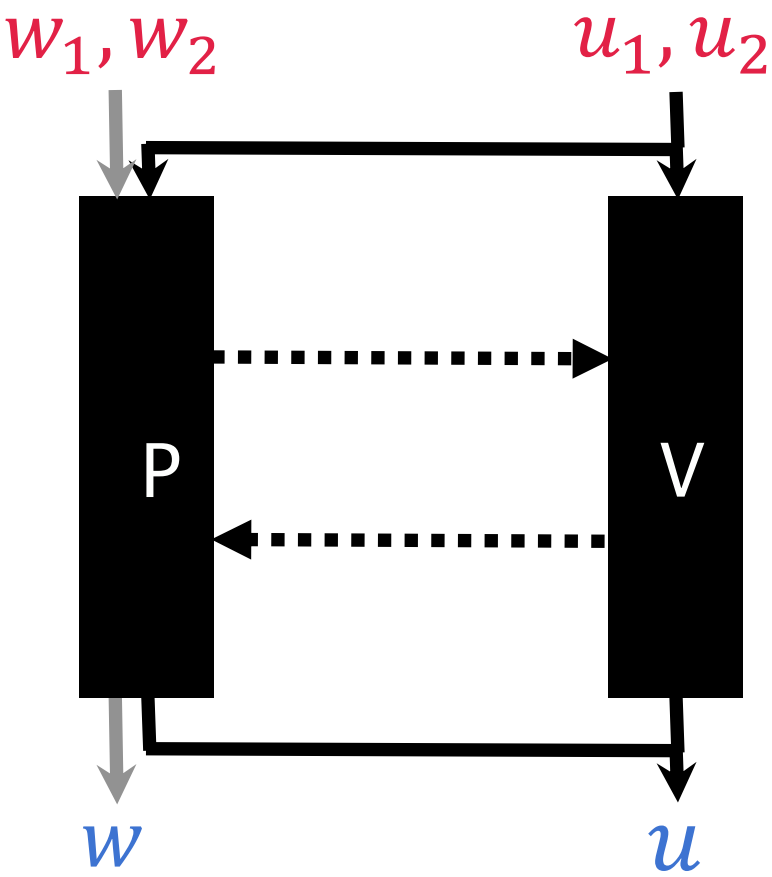
Relaxed Requirement: Interactive Folding Scheme

A folding scheme interactively reduces the claims $(u_1, w_1), (u_2, w_2) \in R$ to a claim $(u, w) \in R$



Relaxed Requirement: Interactive Folding Scheme

A folding scheme interactively reduces the claims $(u_1, w_1), (u_2, w_2) \in R$ to a claim $(u, w) \in R$

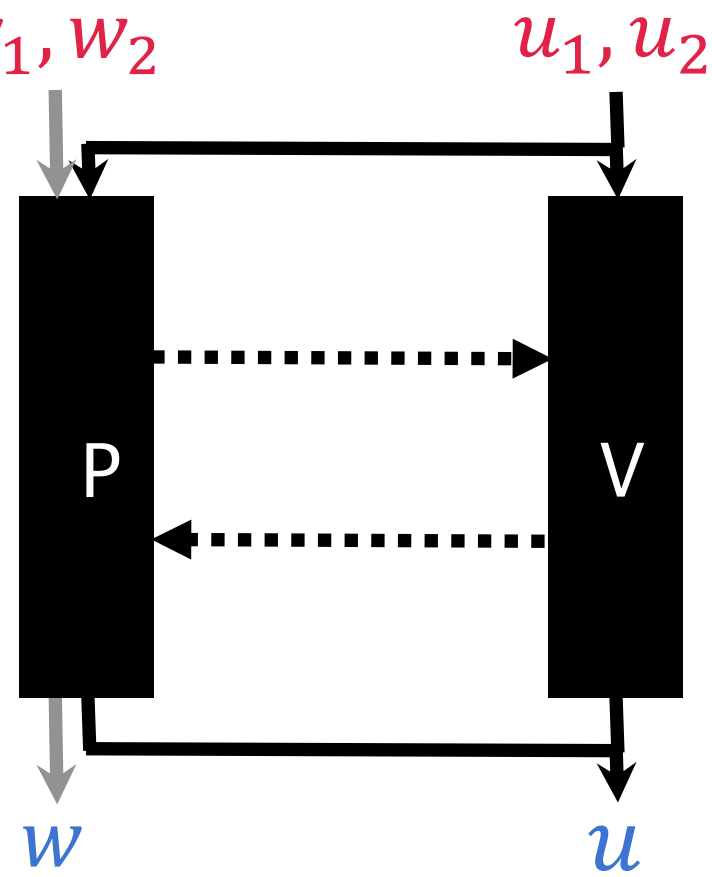


Completeness

If u_1, u_2 are satisfiable then u is satisfiable

Relaxed Requirement: Interactive Folding Scheme

A folding scheme interactively reduces the claims $(u_1, w_1), (u_2, w_2) \in R$ to a claim $(u, w) \in R$



Completeness

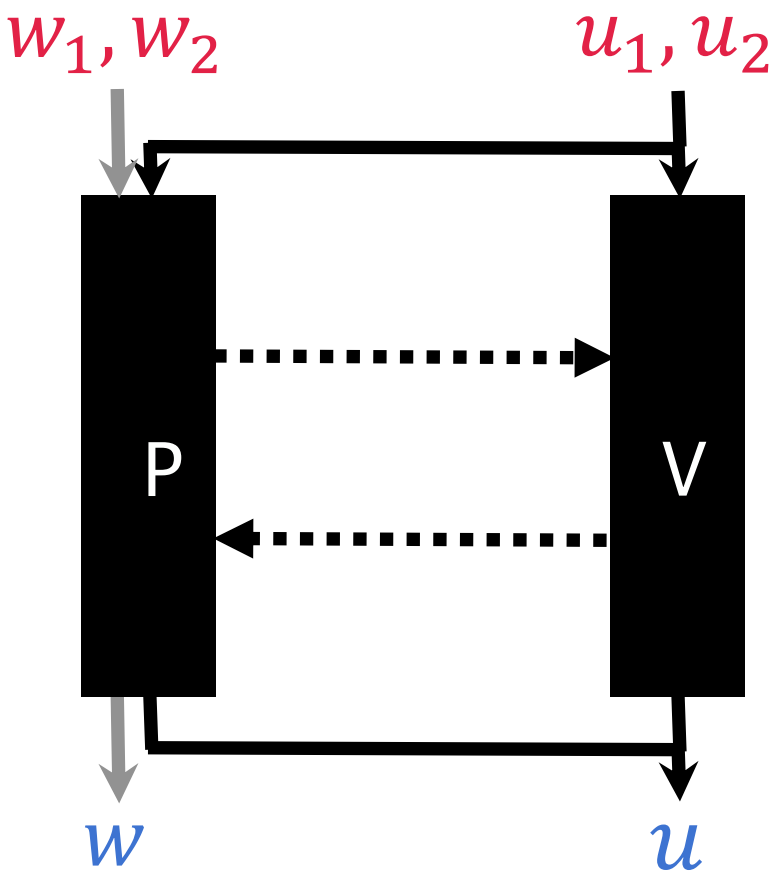
If u_1, u_2 are satisfiable then u is satisfiable

Knowledge Soundness

If prover outputs satisfying w then it must know satisfying w_1, w_2

Relaxed Requirement: Interactive Folding Scheme

A folding scheme interactively reduces the claims $(u_1, w_1), (u_2, w_2) \in R$ to a claim $(u, w) \in R$



Completeness

If u_1, u_2 are satisfiable then u is satisfiable

Knowledge Soundness

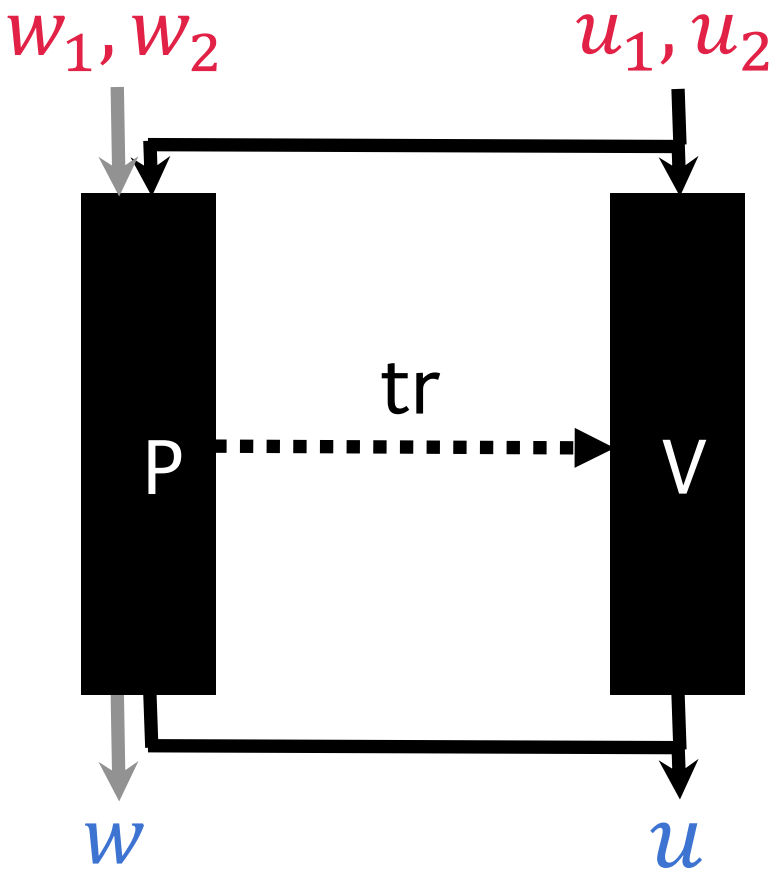
If prover outputs satisfying w then it must know satisfying w_1, w_2

Succinctness

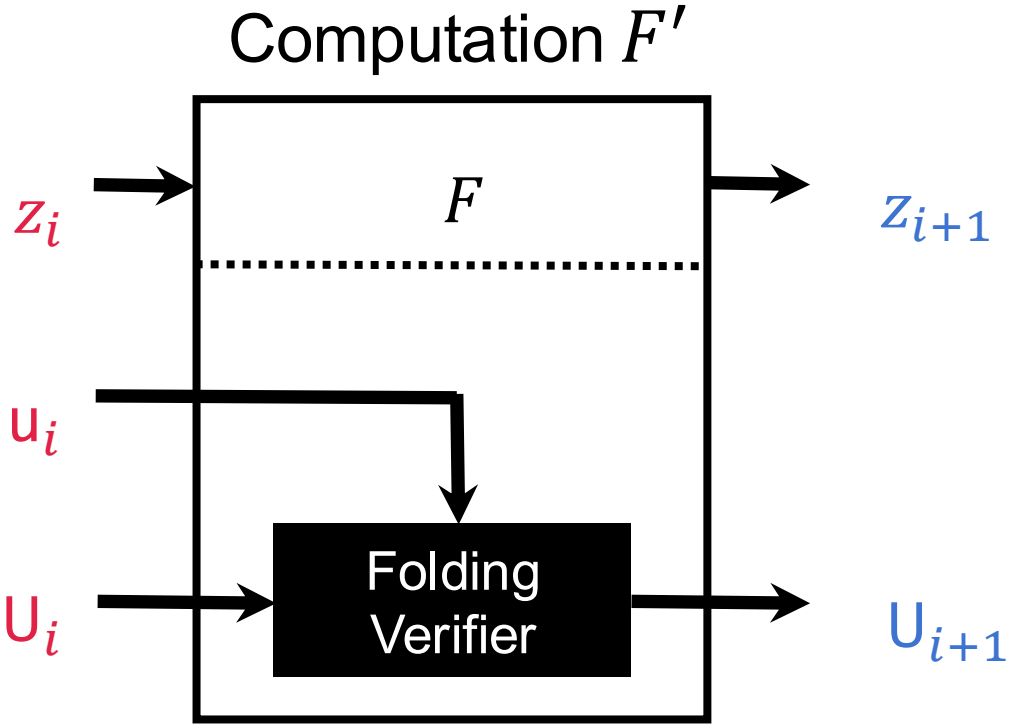
Folding should be more efficient than checking an instance

Non-Interactive Folding Schemes

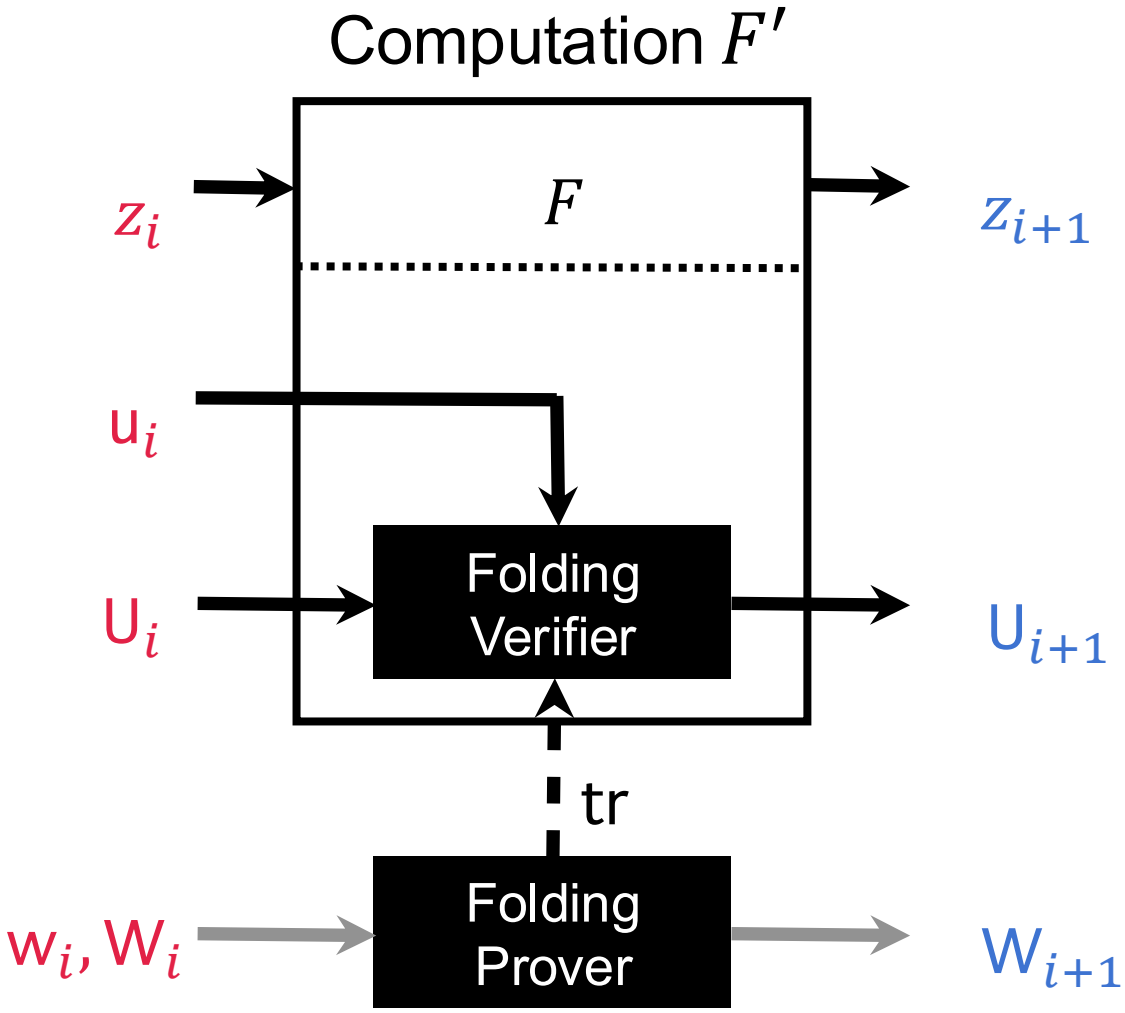
A public-coin folding scheme can be made non-interactive via the Fiat-Shamir Transform.



Given a non-interactive folding scheme for NP, F' can verifiably fold u_i and U_i by running the folding verifier.

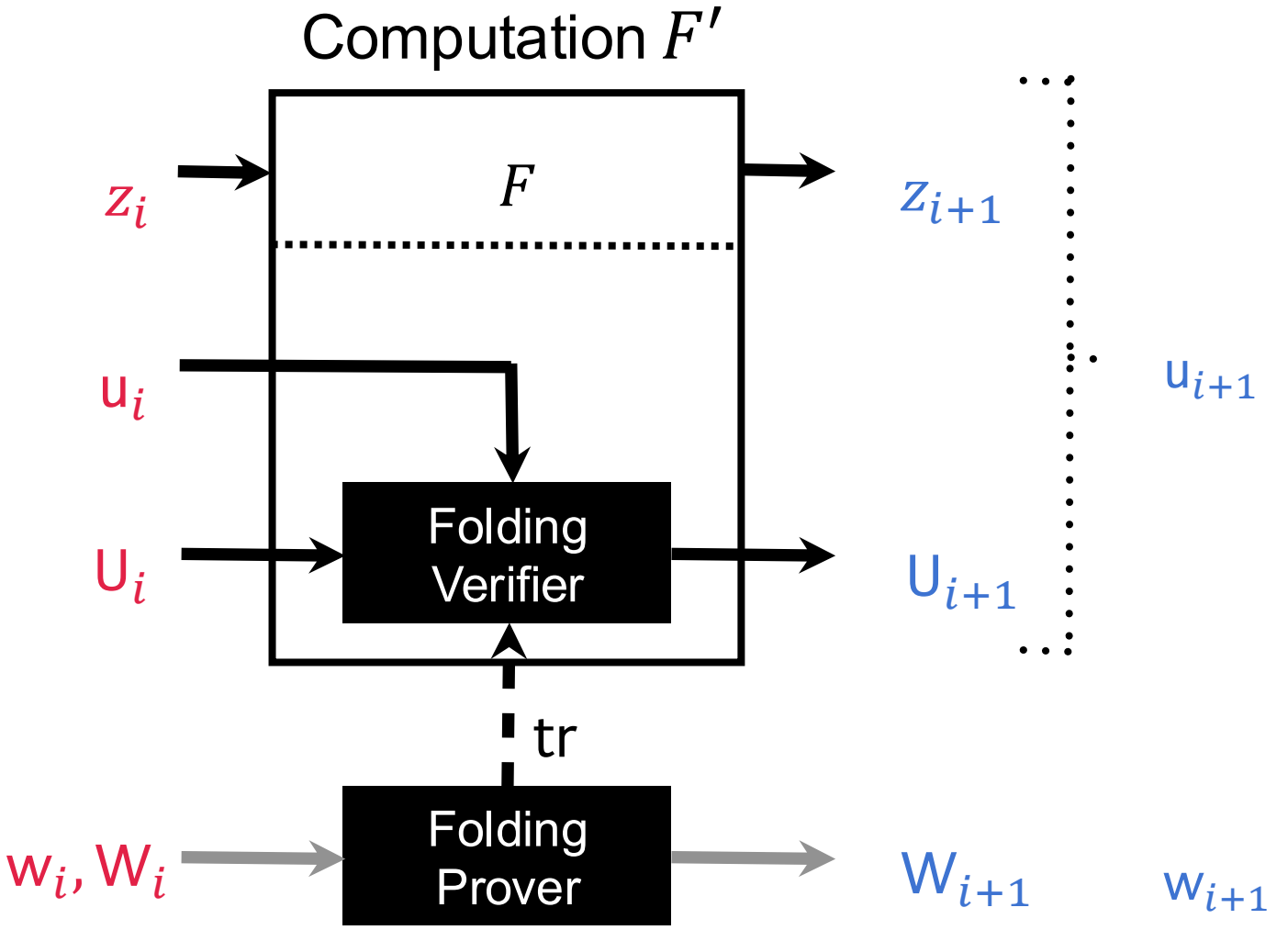


Given a non-interactive folding scheme for NP, F' can verifiably fold u_i and U_i by running the folding verifier.



Witness w_i consists of a trace of the execution of $F(z_{i-1}) = z_i$

Given a non-interactive folding scheme for NP, F' can verifiably fold u_i and U_i by running the folding verifier.



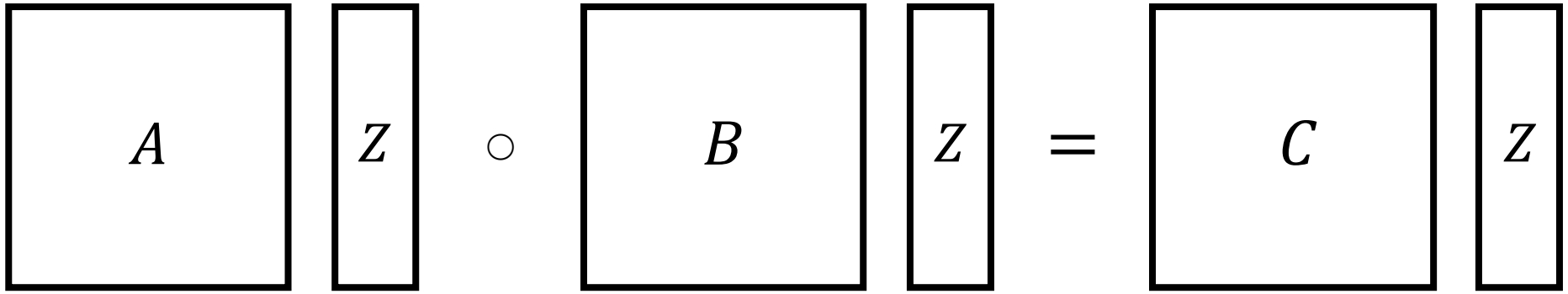
Claim of $F'(z_i) = z_{i+1}$

Witness w_i consists of a trace of the execution of $F(z_{i-1}) = z_i$

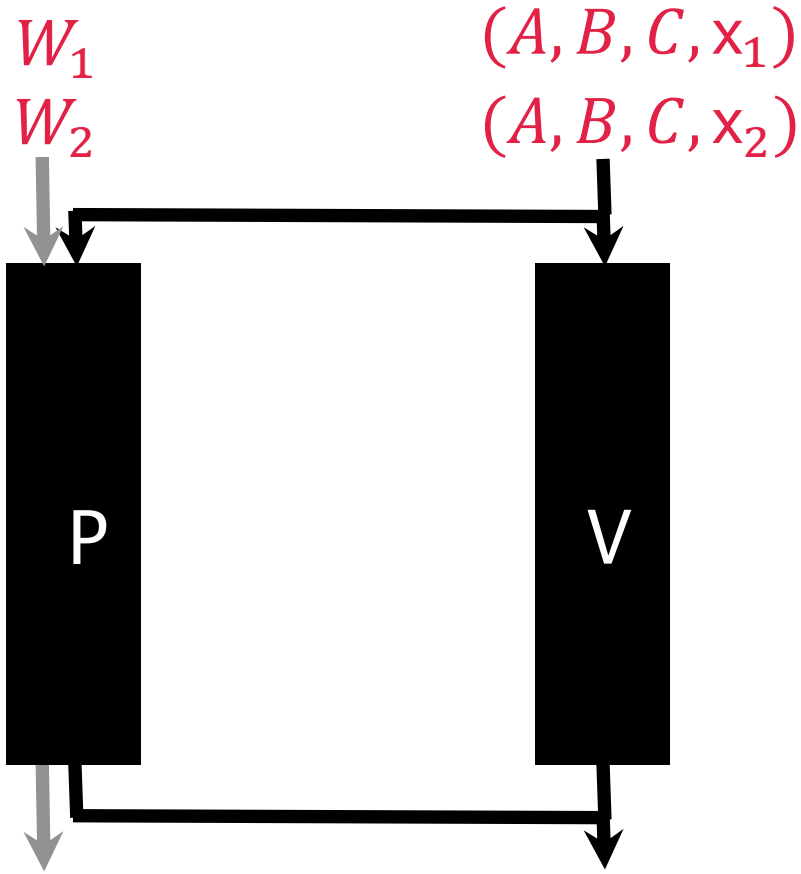
Folding an NP-Complete Relation

We start with R1CS

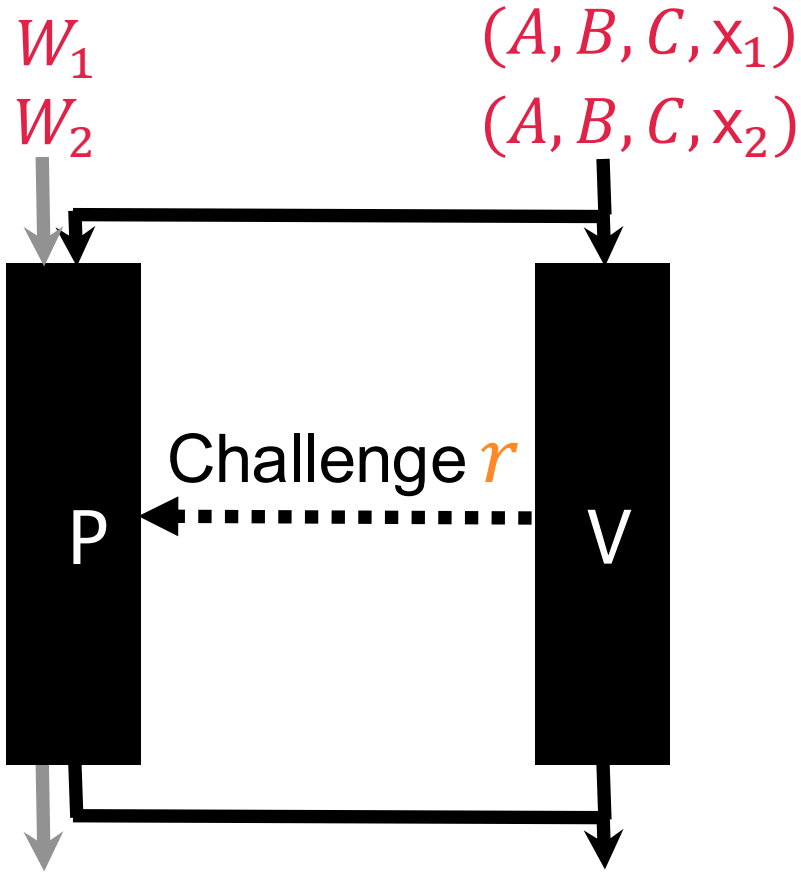
An R1CS statement consists of constraint matrices A, B, C, and vector x. A witness vector W is satisfying if for $Z = (W, \mathrm{x}, 1)$



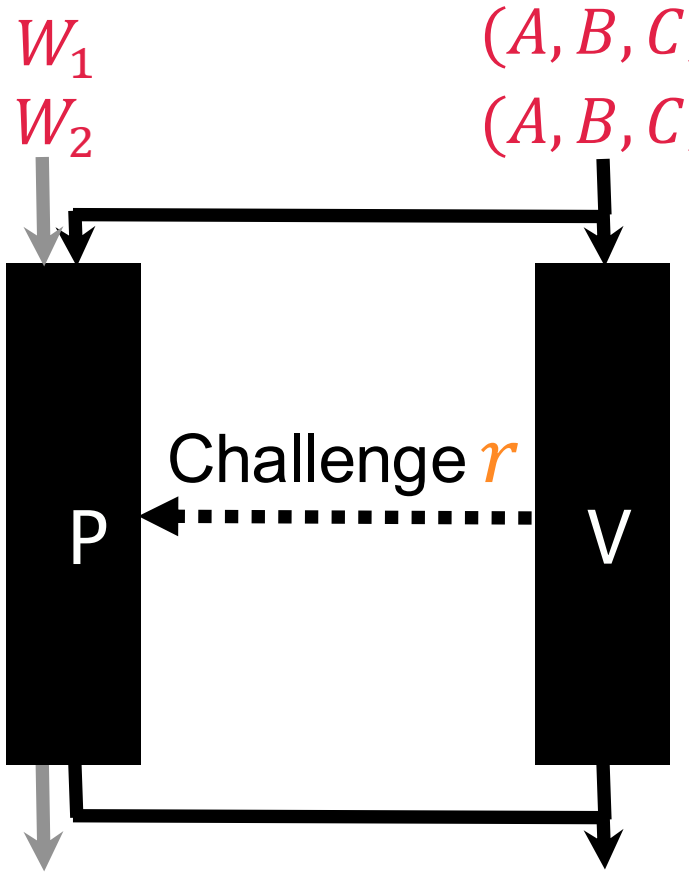
Attempt to Fold R1CS Instances



Attempt to Fold R1CS Instances

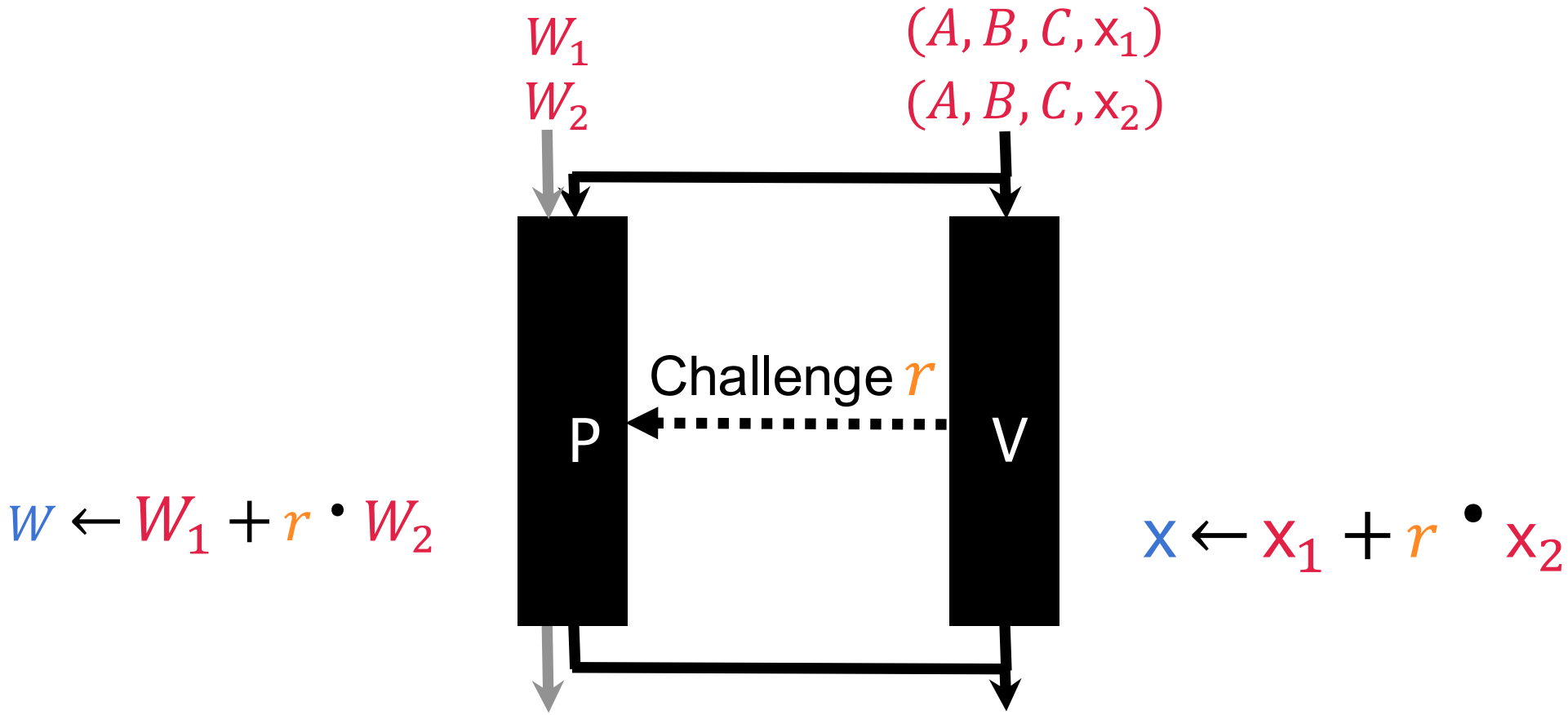


Attempt to Fold R1CS Instances

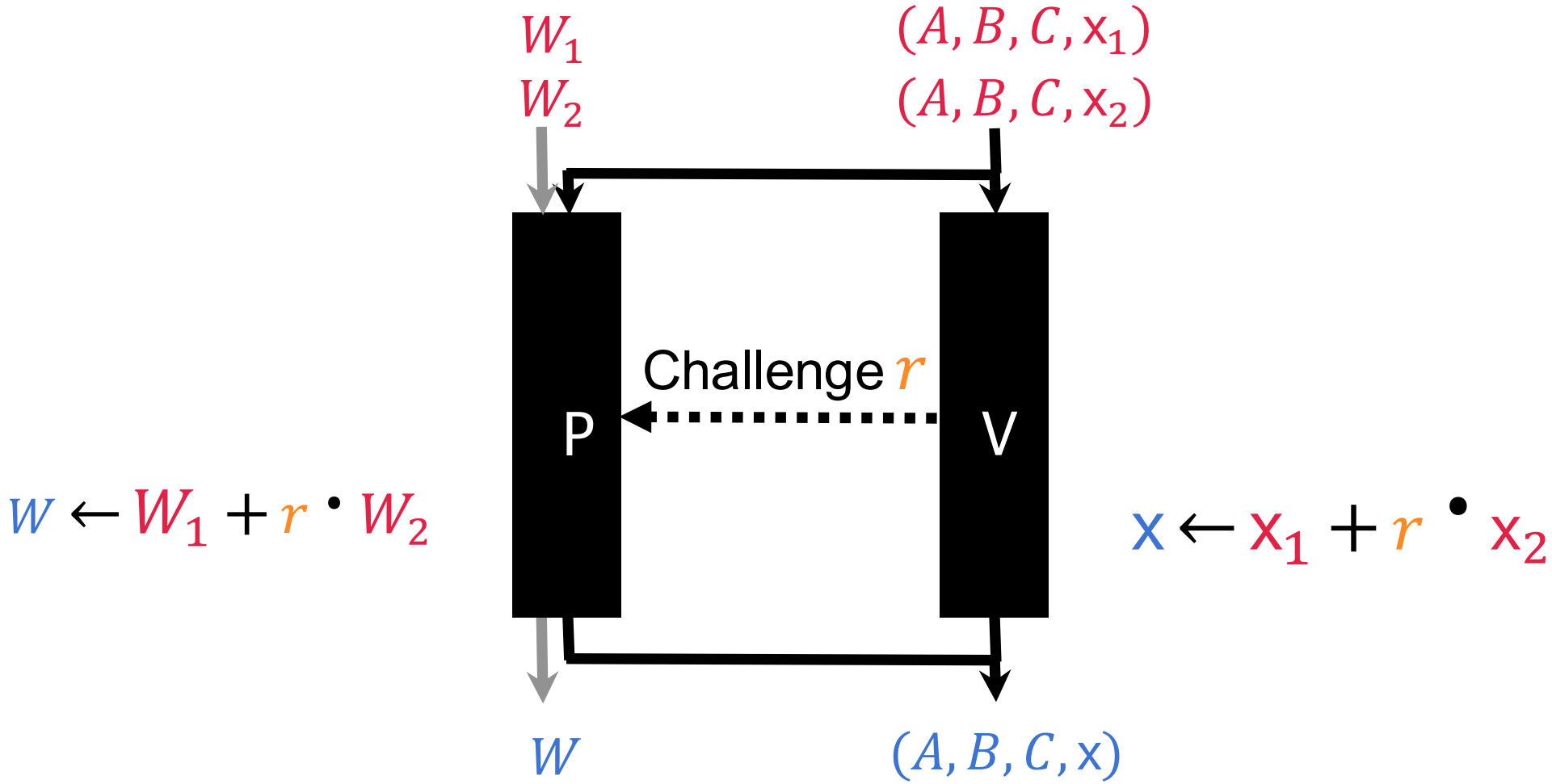


$$\mathrm{x} \leftarrow \mathrm{x}_1 + r \cdot \mathrm{x}_2$$

Attempt to Fold R1CS Instances



Attempt to Fold R1CS Instances



Attempt to Fold R1CS Instances

Unfortunately, letting $Z_i = (W_i, \mathrm{x}_i, 1)$ and $Z = Z_1 + r \cdot Z_2$, we have that $AZ \circ BZ \neq CZ$

Attempt to Fold R1CS Instances

Unfortunately, letting $Z_i = (W_i, \mathrm{x}_i, 1)$ and $Z = Z_1 + r \cdot Z_2$, we have that $AZ \circ BZ \neq CZ$

$$CZ = AZ_1 \circ BZ_1 + r \cdot AZ_2 \circ BZ_2$$

Attempt to Fold R1CS Instances

Unfortunately, letting $Z_i = (W_i, \mathrm{x}_i, 1)$ and $Z = Z_1 + r \cdot Z_2$, we have that $AZ \circ BZ \neq CZ$

$$CZ = AZ_1 \circ BZ_1 + r \cdot AZ_2 \circ BZ_2$$

$$AZ \circ BZ = AZ_1 \circ BZ_1 + r \cdot (AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1) + r^2 \cdot AZ_2 \circ BZ_2$$

Attempt to Fold R1CS Instances

Unfortunately, letting $Z_i = (W_i, \mathrm{x}_i, 1)$ and $Z = Z_1 + r \cdot Z_2$, we have that $AZ \circ BZ \neq CZ$

$$CZ = AZ_1 \circ BZ_1 + r \cdot AZ_2 \circ BZ_2$$

$$AZ \circ BZ = AZ_1 \circ BZ_1 + r \cdot (AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1) + r^2 \cdot AZ_2 \circ BZ_2$$

To absorb the error terms we introduce an error vector E in the statement

Attempt to Fold R1CS Instances

Unfortunately, letting $Z_i = (W_i, \mathrm{x}_i, 1)$ and $Z = Z_1 + r \cdot Z_2$, we have that $AZ \circ BZ \neq CZ$

$$CZ = AZ_1 \circ BZ_1 + r \cdot AZ_2 \circ BZ_2$$

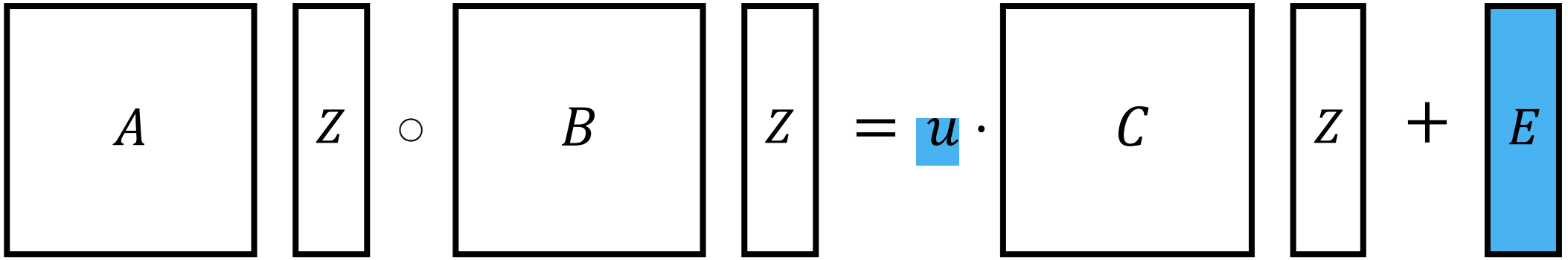
To absorb the extra r factor we introduce scalar u in the statement

$$AZ \circ BZ = AZ_1 \circ BZ_1 + r \cdot (AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1) + r^2 \cdot AZ_2 \circ BZ_2$$

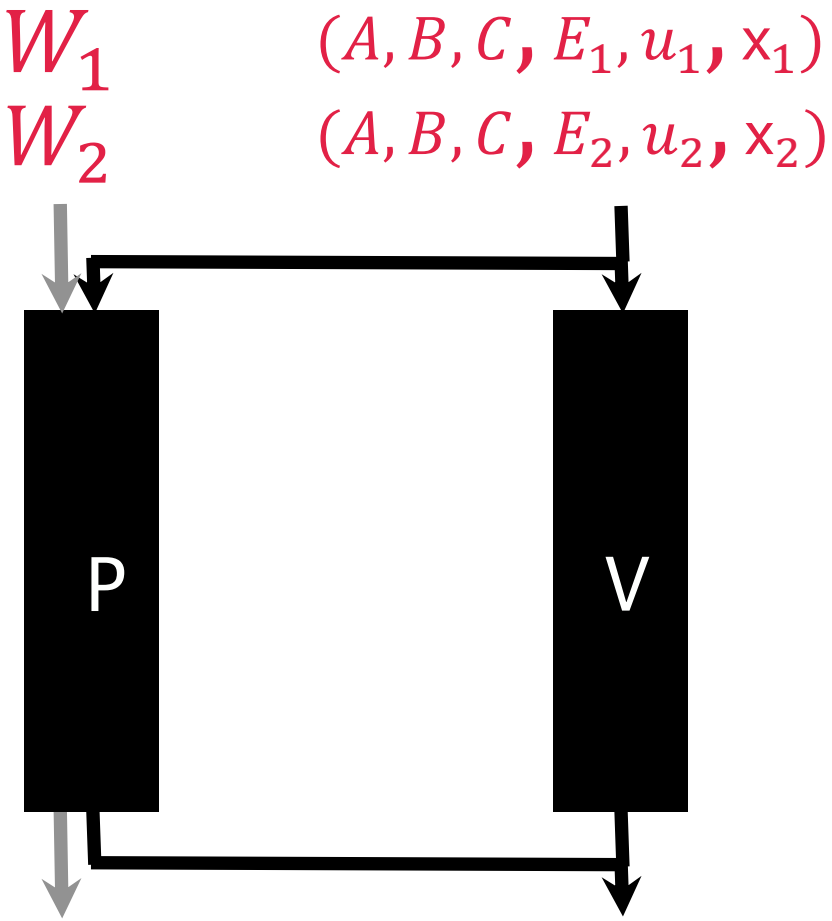
To absorb the error terms we introduce an error vector E in the statement

Relaxed R1CS: A Variant of R1CS Amenable to Folding

A relaxed R1CS statement additionally contains an error vector E and scalar u. A witness vector W is satisfying if for $Z = (W, \mathrm{x}, u)$

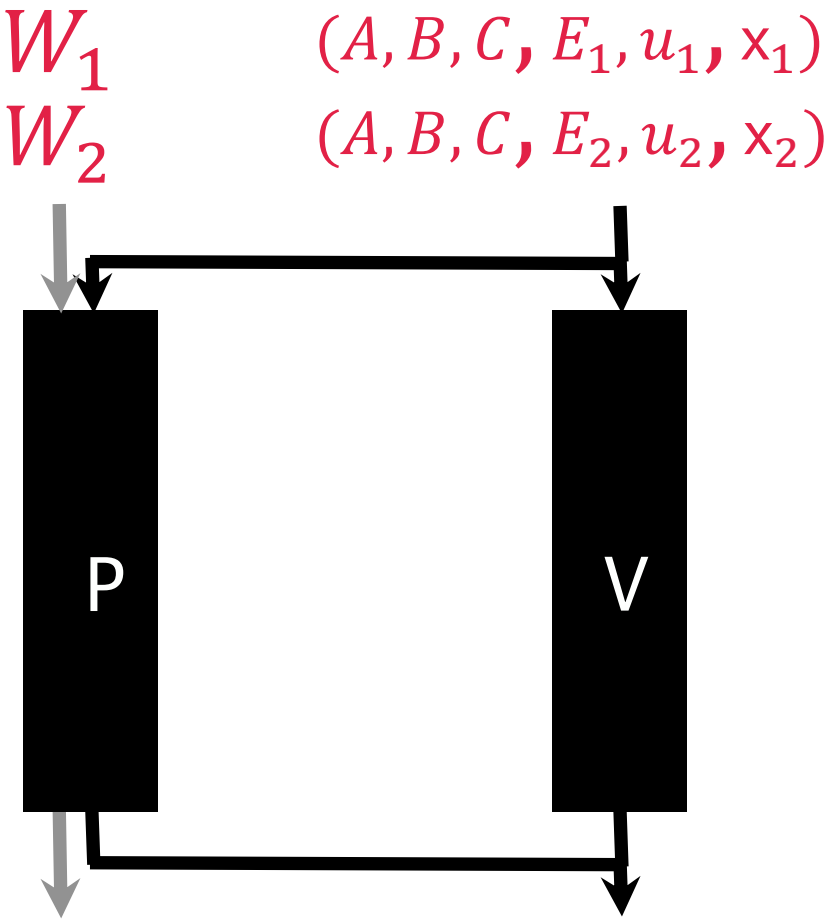


Folding Scheme for Relaxed R1CS (Almost)



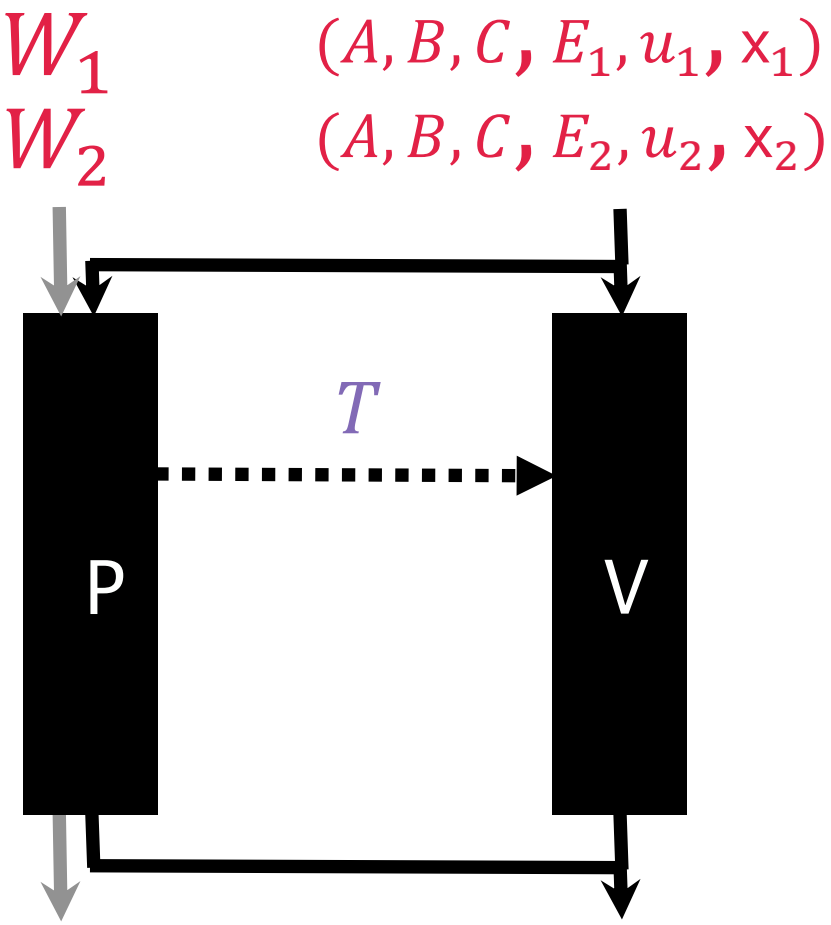
Folding Scheme for Relaxed R1CS (Almost)

$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$



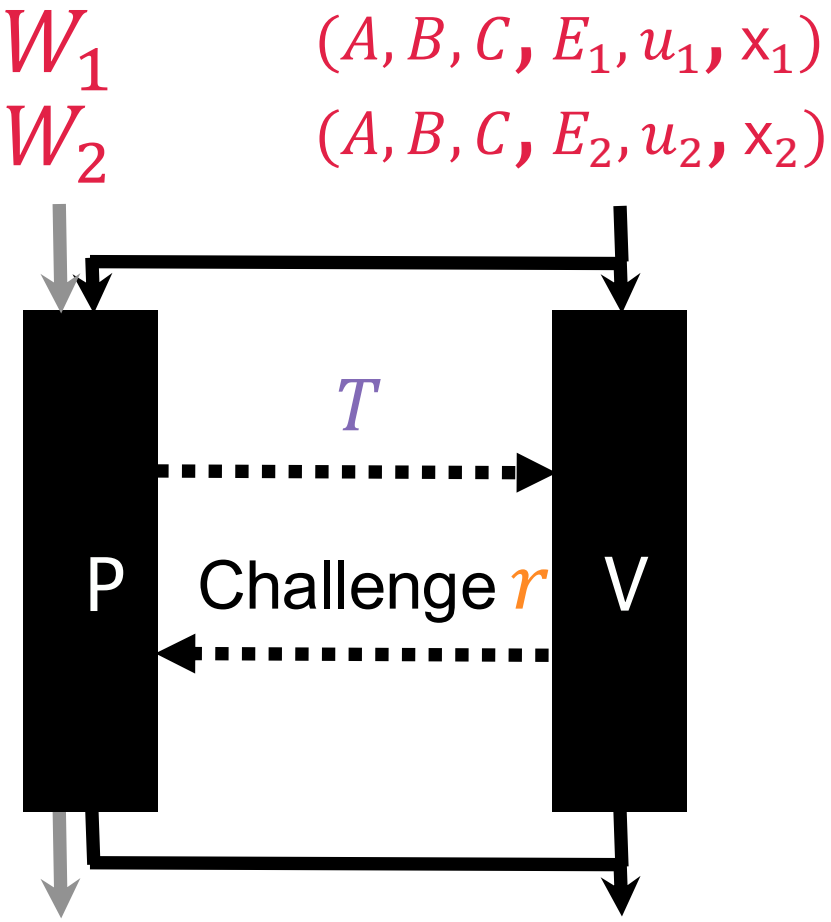
Folding Scheme for Relaxed R1CS (Almost)

$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$



Folding Scheme for Relaxed R1CS (Almost)

$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$

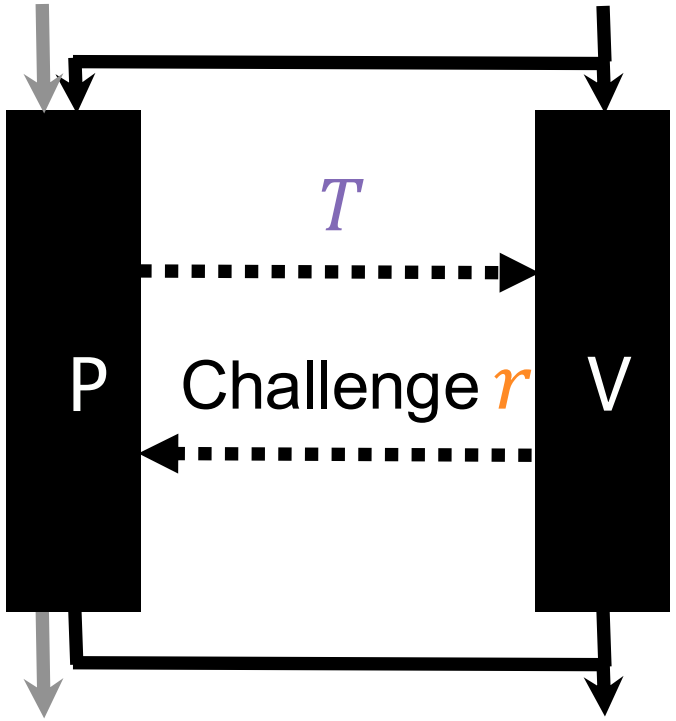


Folding Scheme for Relaxed R1CS (Almost)

W_1 $(A, B, C, E_1, u_1, \mathrm{x}_1)$

W_2 $(A, B, C, E_2, u_2, \mathrm{x}_2)$

$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$



$u \leftarrow u_1 + r \cdot u_2$

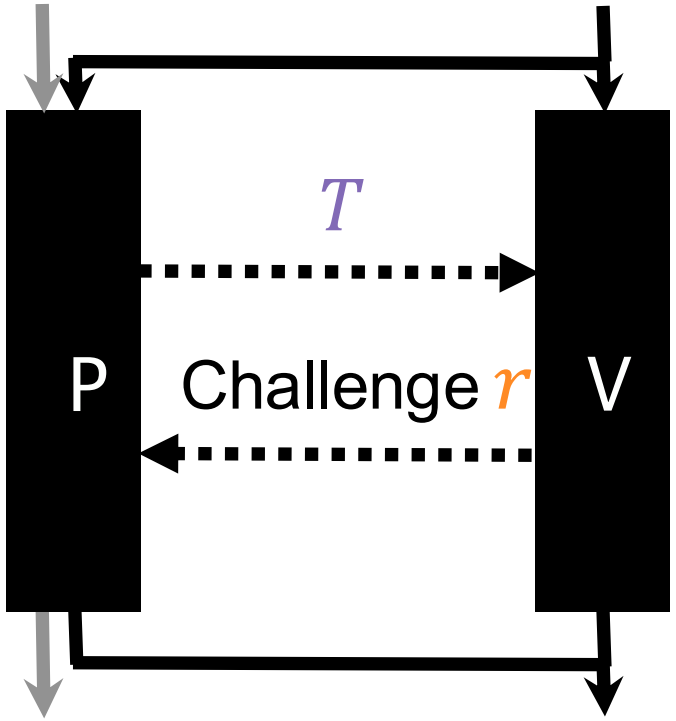
$\mathrm{x} \leftarrow \mathrm{x}_1 + r \cdot \mathrm{x}_2$

Folding Scheme for Relaxed R1CS (Almost)

W_1 $(A, B, C, E_1, u_1, \mathrm{x}_1)$

W_2 $(A, B, C, E_2, u_2, \mathrm{x}_2)$

$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$

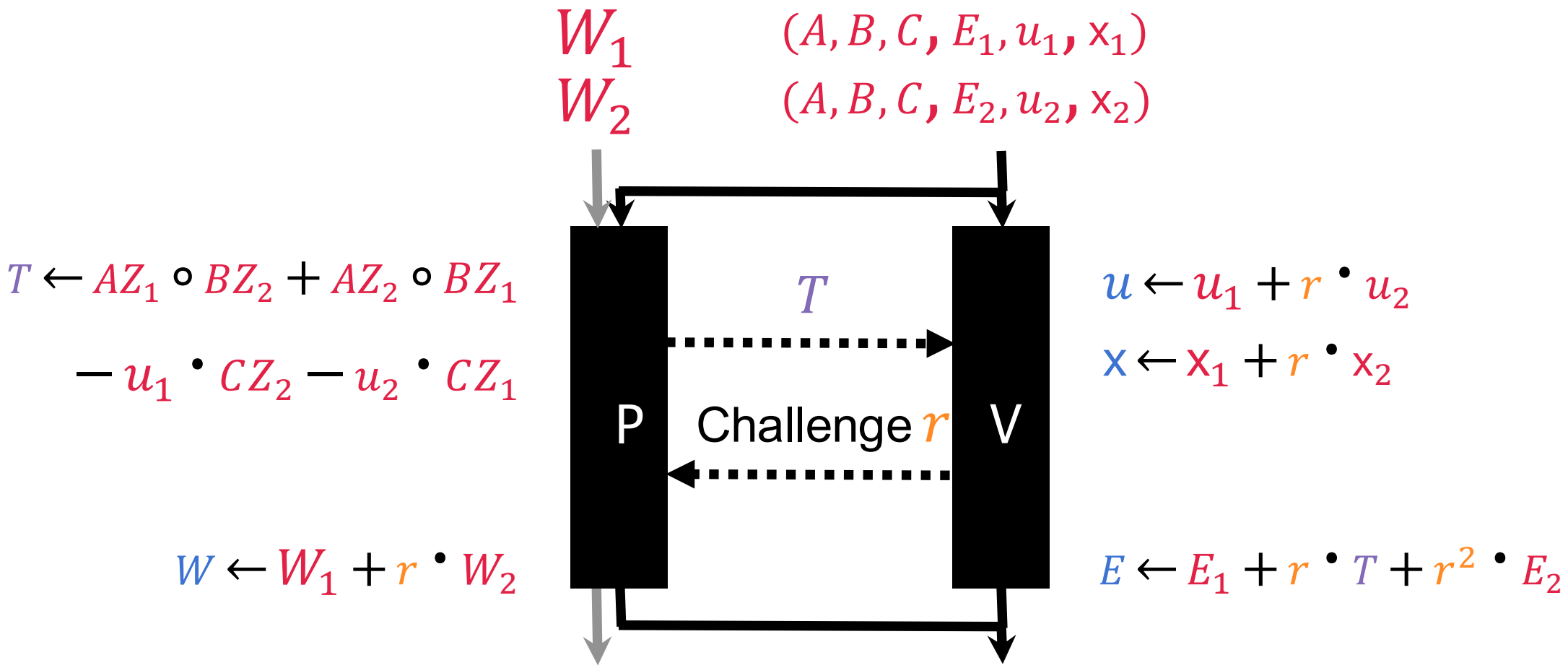


$u \leftarrow u_1 + r \cdot u_2$

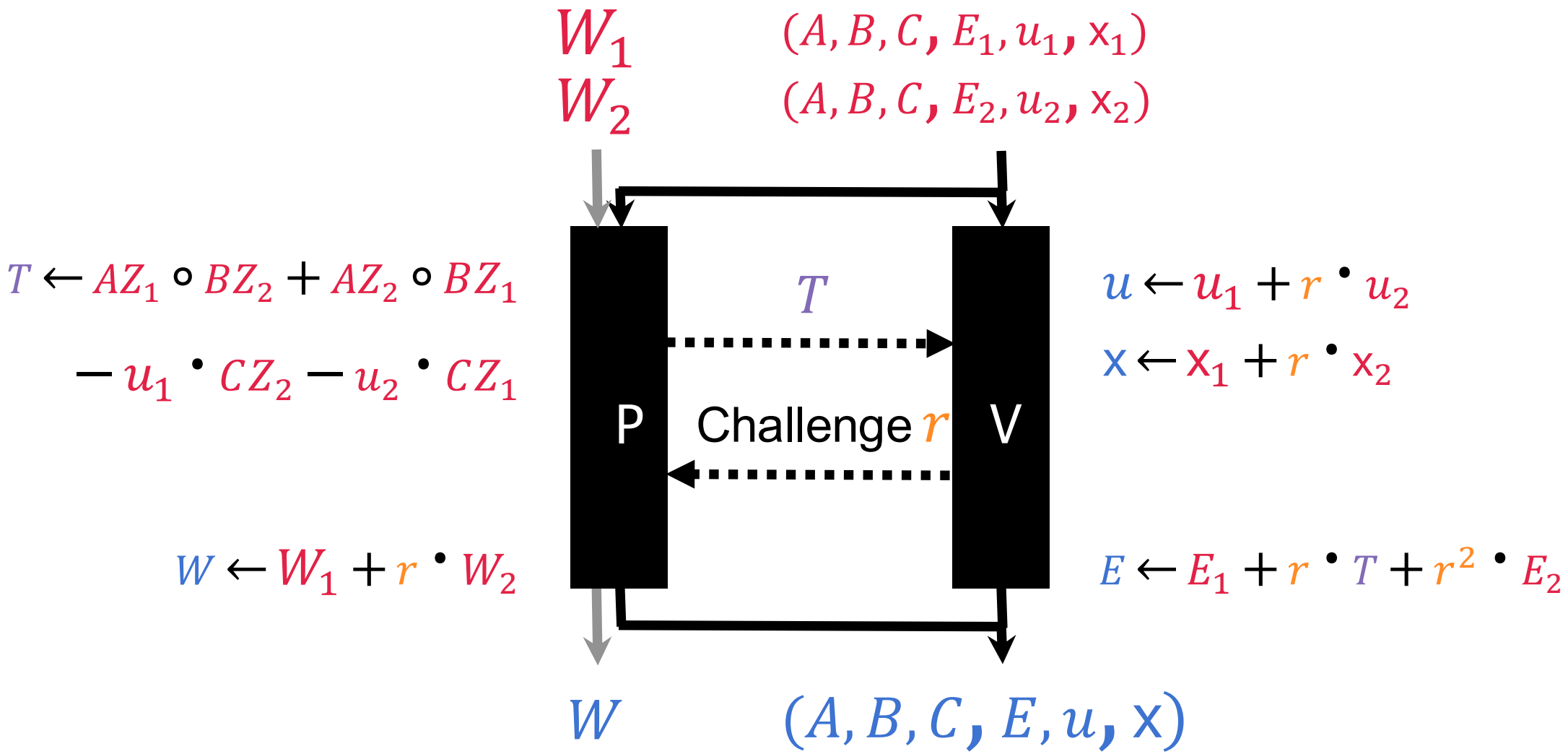
$\mathrm{x} \leftarrow \mathrm{x}_1 + r \cdot \mathrm{x}_2$

$E \leftarrow E_1 + r \cdot T + r^2 \cdot E_2$

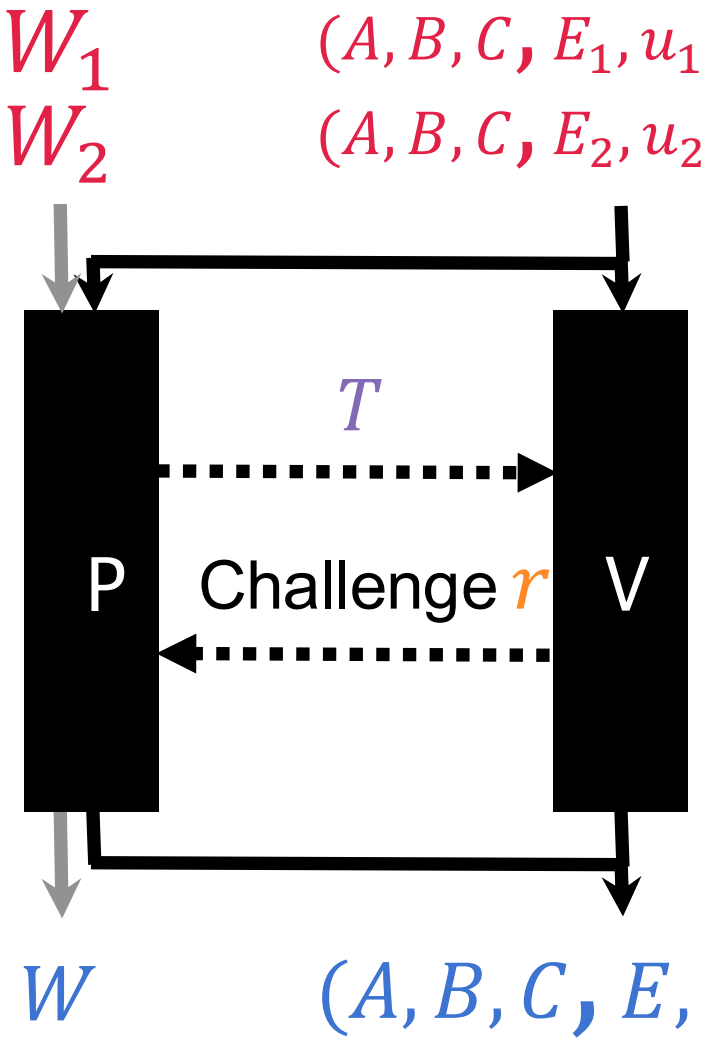
Folding Scheme for Relaxed R1CS (Almost)



Folding Scheme for Relaxed R1CS (Almost)



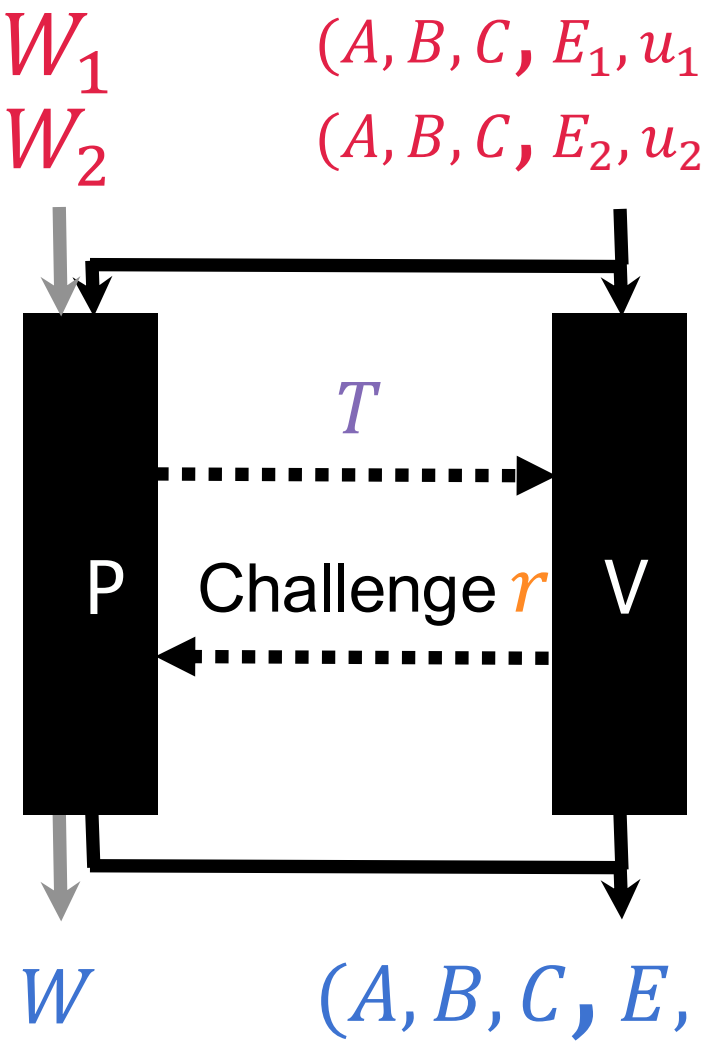
Folding Scheme for Relaxed R1CS (Almost)



Problem: Verifier cannot enforce that prover folds W correctly

Folding Scheme for Relaxed R1CS (Almost)

Problem:
Because $|E| = O(|W|)$ the verifier is not succinct



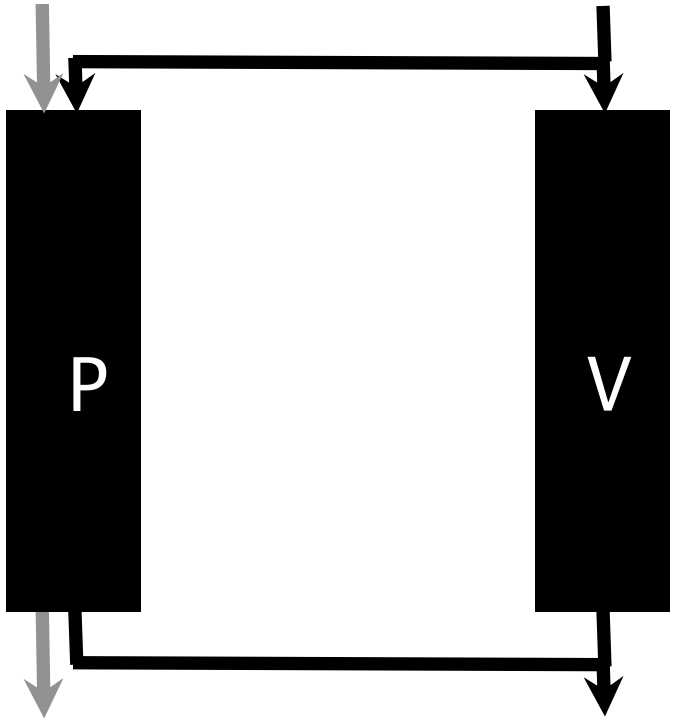
Problem: Verifier cannot enforce that prover folds W correctly

Folding Scheme for Relaxed R1CS from Additive Commitments

Solution: Treat (E, W) as the witness with commit in statement

(E_1, W_1) $(A, B, C, \overline{E}_1, \overline{W}_1, u_1, \mathrm{x}_1)$

(E_2, W_2) $(A, B, C, \overline{E}_2, \overline{W}_2, u_2, \mathrm{x}_2)$



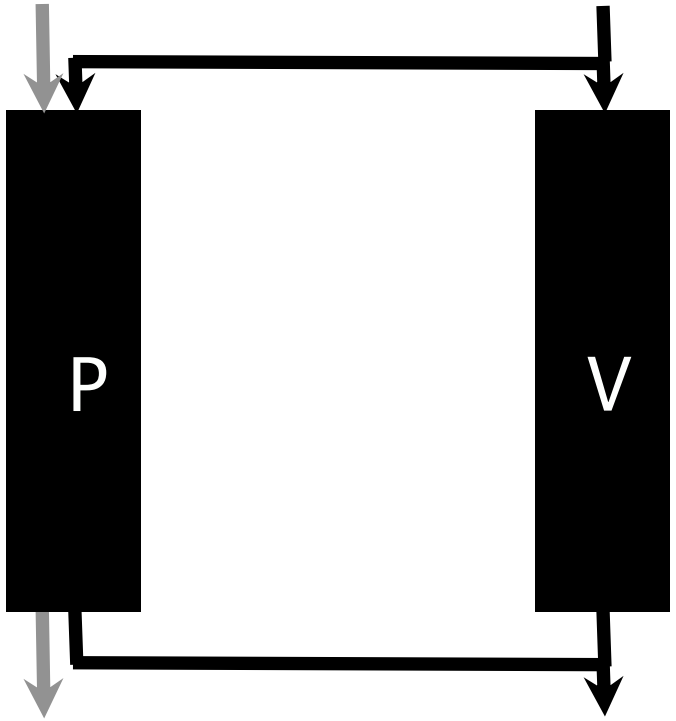
Folding Scheme for Relaxed R1CS from Additive Commitments

Solution: Treat (E, W) as the witness with commit in statement

$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$

(E_1, W_1) $(A, B, C, \overline{E}_1, \overline{W}_1, u_1, \mathrm{x}_1)$

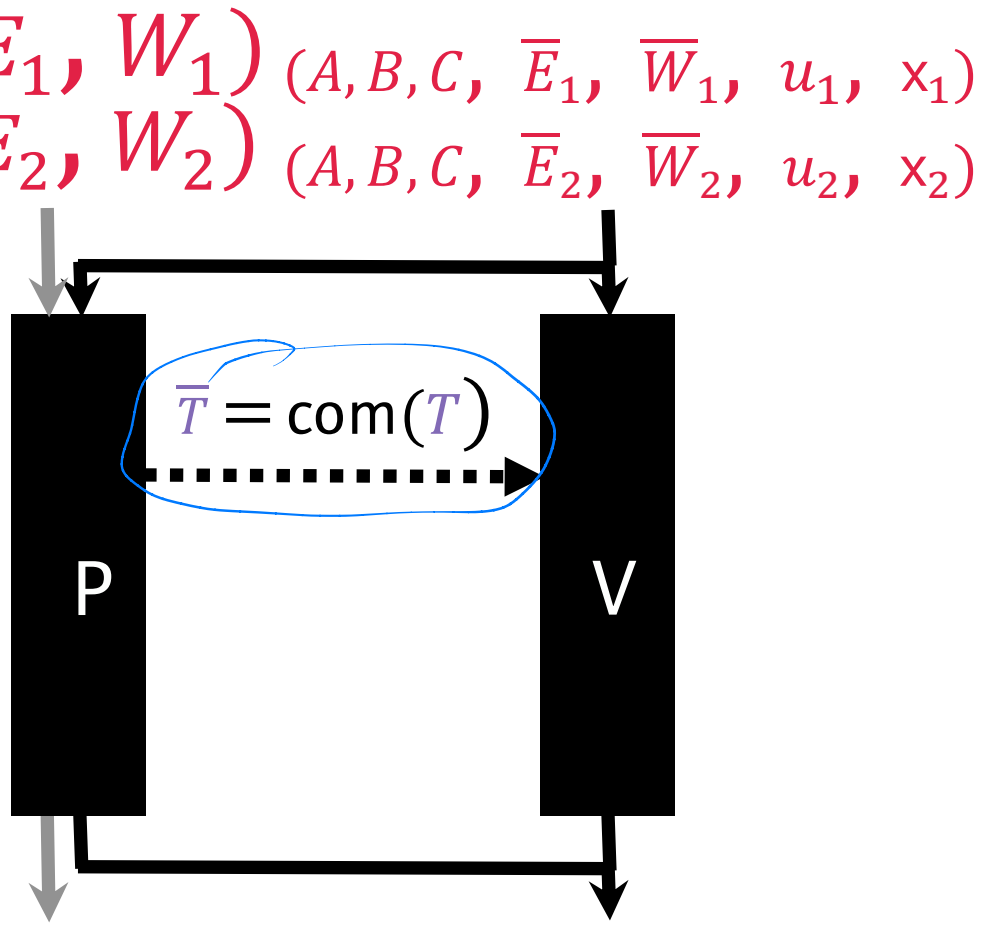
(E_2, W_2) $(A, B, C, \overline{E}_2, \overline{W}_2, u_2, \mathrm{x}_2)$



Folding Scheme for Relaxed R1CS from Additive Commitments

Solution: Treat (E, W) as the witness with commit in statement

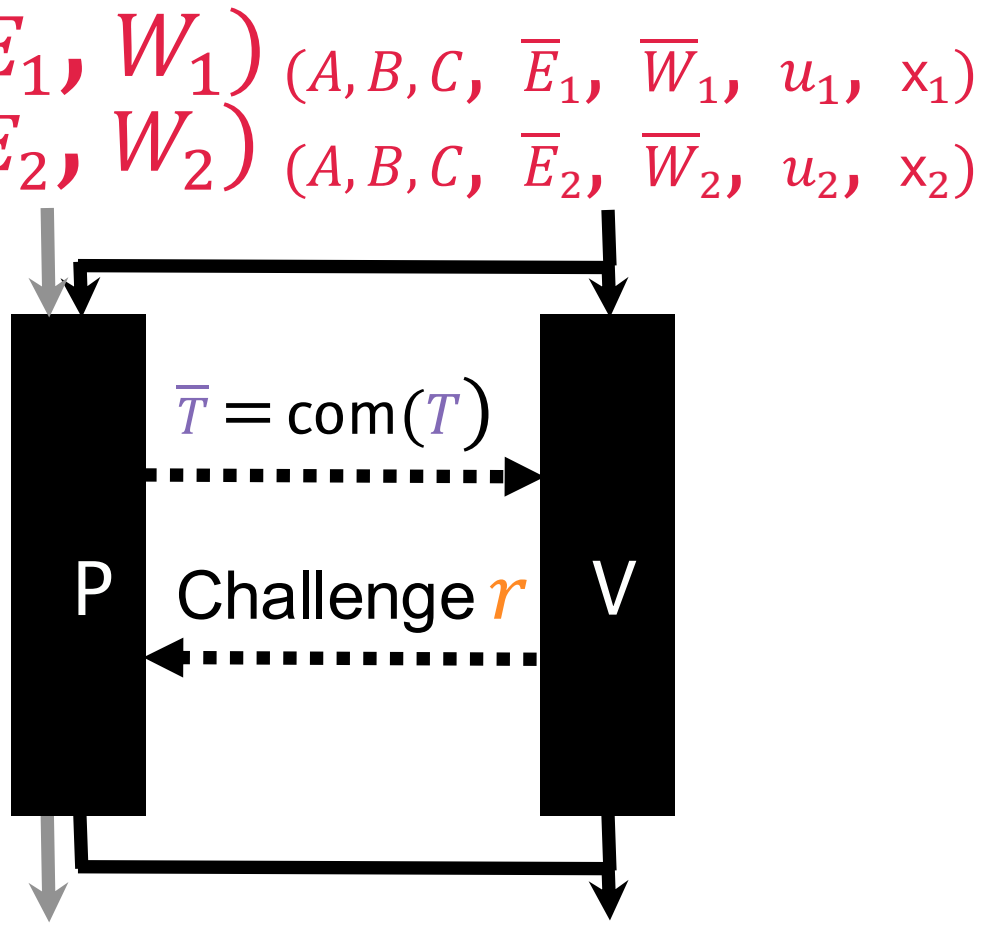
$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$



Folding Scheme for Relaxed R1CS from Additive Commitments

Solution: Treat (E, W) as the witness with commit in statement

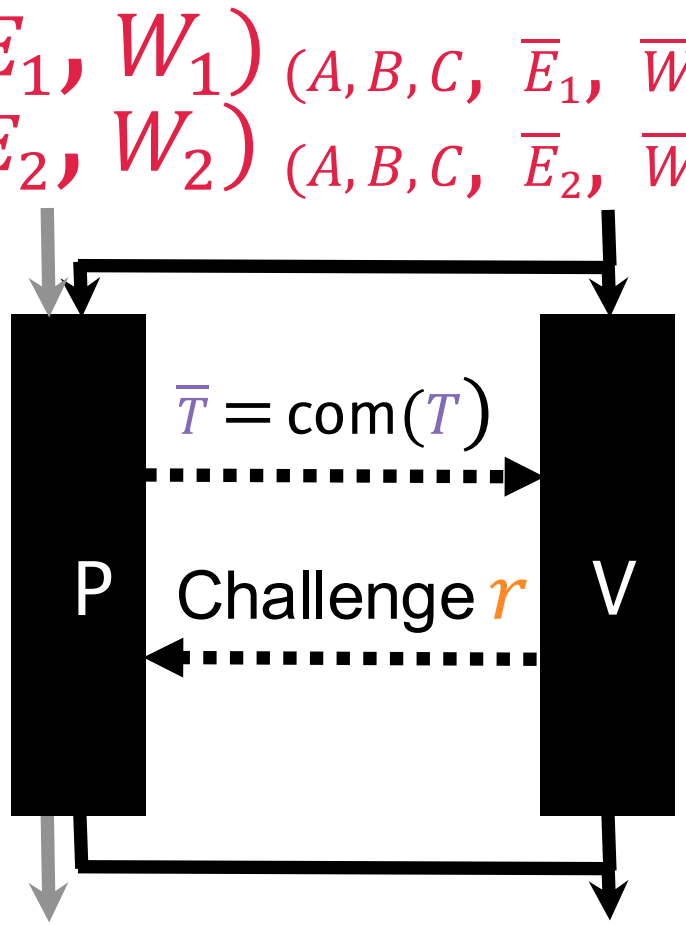
$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$



Folding Scheme for Relaxed R1CS from Additive Commitments

Solution: Treat (E, W) as the witness with commit in statement

$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$



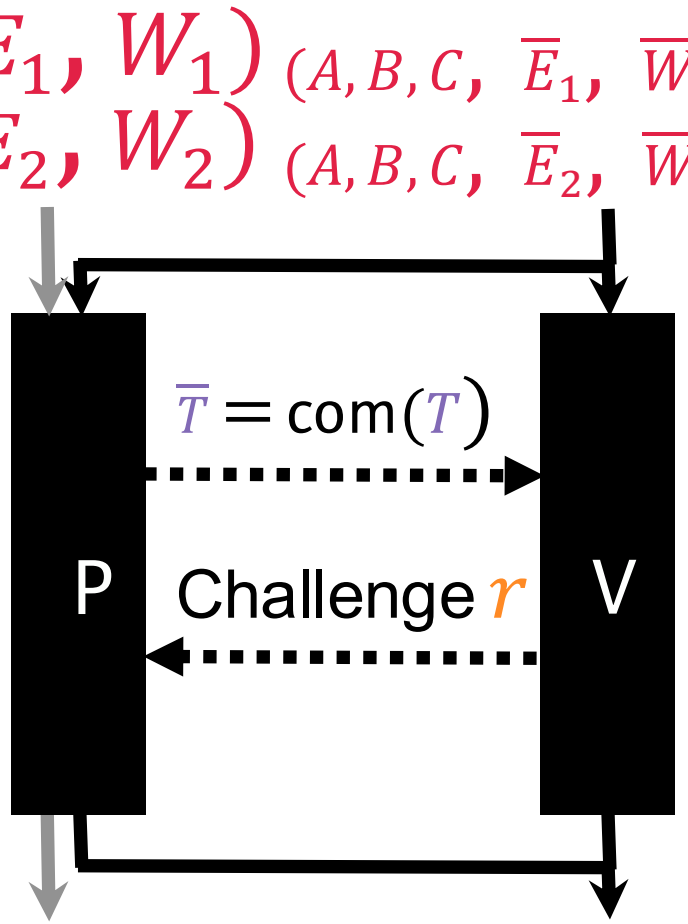
$$u \leftarrow u_1 + r \cdot u_2$$

$$\mathrm{x} \leftarrow \mathrm{x}_1 + r \cdot \mathrm{x}_2$$

Folding Scheme for Relaxed R1CS from Additive Commitments

Solution: Treat (E, W) as the witness with commit in statement

$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$



$$u \leftarrow u_1 + r \cdot u_2$$

$$\mathrm{x} \leftarrow \mathrm{x}_1 + r \cdot \mathrm{x}_2$$

$$\overline{W} \leftarrow \overline{W}_1 + r \cdot \overline{W}_2$$

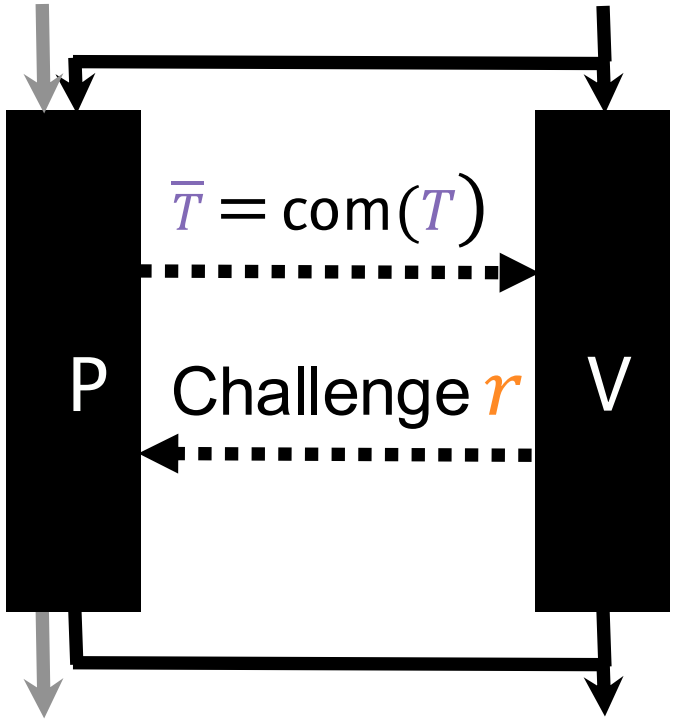
$$\overline{E} \leftarrow \overline{E}_1 + r \cdot \overline{T} + r^2 \cdot \overline{E}_2$$

Folding Scheme for Relaxed R1CS from Additive Commitments

Solution: Treat (E, W) as the witness with commit in statement

(E_1, W_1) $(A, B, C, \overline{E}_1, \overline{W}_1, u_1, \mathrm{x}_1)$

(E_2, W_2) $(A, B, C, \overline{E}_2, \overline{W}_2, u_2, \mathrm{x}_2)$



$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$

$$W \leftarrow W_1 + r \cdot W_2$$

$$E \leftarrow E_1 + r \cdot T + r^2 \cdot E_2$$

$$u \leftarrow u_1 + r \cdot u_2$$

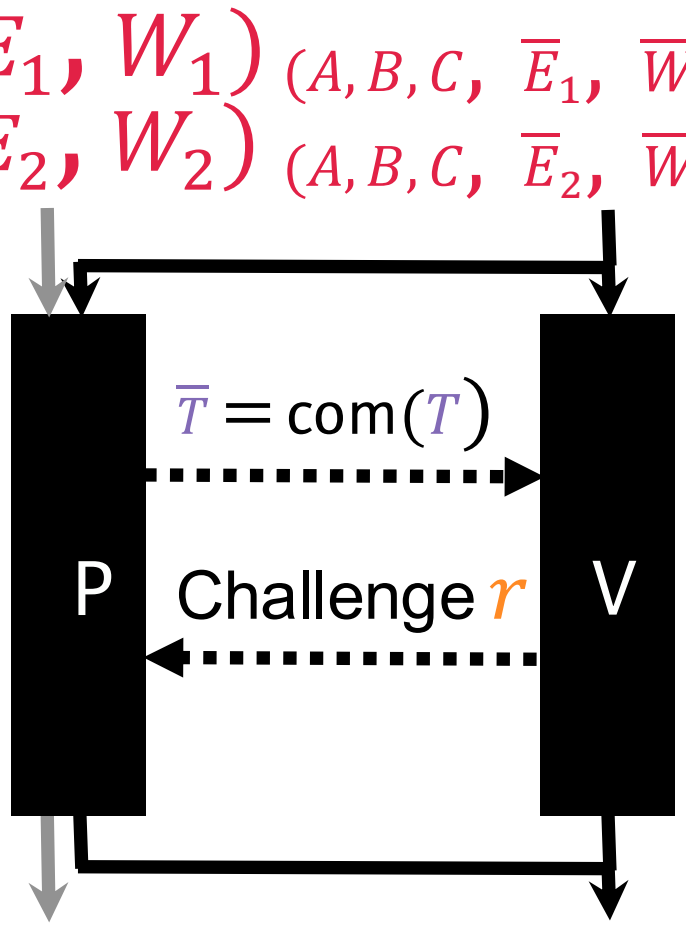
$$\mathrm{x} \leftarrow \mathrm{x}_1 + r \cdot \mathrm{x}_2$$

$$\overline{W} \leftarrow \overline{W}_1 + r \cdot \overline{W}_2$$

$$\overline{E} \leftarrow \overline{E}_1 + r \cdot \overline{T} + r^2 \cdot \overline{E}_2$$

Folding Scheme for Relaxed R1CS from Additive Commitments

Solution: Treat (E, W) as the witness with commit in statement



$$T \leftarrow AZ_1 \circ BZ_2 + AZ_2 \circ BZ_1 - u_1 \cdot CZ_2 - u_2 \cdot CZ_1$$

$$W \leftarrow W_1 + r \cdot W_2$$

$$E \leftarrow E_1 + r \cdot T + r^2 \cdot E_2$$

$$u \leftarrow u_1 + r \cdot u_2$$

$$\mathrm{x} \leftarrow \mathrm{x}_1 + r \cdot \mathrm{x}_2$$

$$\overline{W} \leftarrow \overline{W}_1 + r \cdot \overline{W}_2$$

$$\overline{E} \leftarrow \overline{E}_1 + r \cdot \overline{T} + r^2 \cdot \overline{E}_2$$

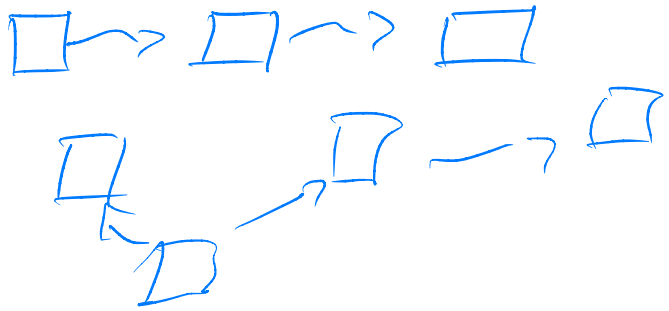
Agenda for this lecture

- Announcements
- Motivating incremental proofs
- Incrementally verifiable computation (IVC) via recursion
- IVC via folding, the Nova construction
- Proof-carrying data

Proof-carrying data (PCD)

(use recursion)

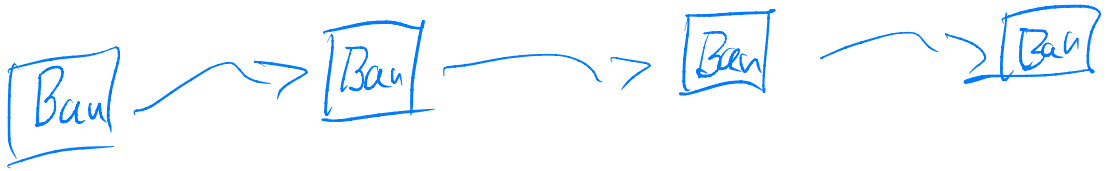
- Proof-carrying data
  IVC but a DAG



- non-uniform IVC
  Run different F functions

"switchboarding"

- Alpaca: anonymous moderation from IVC



- Post-quantum IVC